

A Leakage-Safe Evaluation Pipeline for Heart/Lung Sound Source Separation on HLS-CMDS

[Author Name]
TEEP Internship

2026-08-17, revised 2026-09-13

Abstract

This report documents a software pipeline built to evaluate blind source separation of heart and lung sounds on the HLS-CMDS dataset [1]. The pipeline consists of: (1) a data-loading and integrity-validation layer, including a per-row additive-triplet audit that found only 36 of `Mix.csv`'s 145 rows have a mixed recording that is acoustically related to its named heart/lung sources at all — the other 109 fail even before leakage is considered; (2) a leakage-safe, triplet-level cross-validation split derived from a content-hash audit of the dataset that found a third of the “isolated” ground-truth recordings are literal duplicates of recordings used inside the mixtures; (3) a BSS Eval (SDR/SIR/SAR) metrics module wrapping `mir_eval`, validated both by direct cross-checking against `mir_eval` and by closed-form synthetic tests; (4) a k-fold evaluation harness tying the split and the metrics together with per-fold, across-fold, and pooled row-level aggregation; (5) a combinatorial synthetic mixture set (`synthetic_mix.py`) that supersedes the 36-row native-additive set as the primary evaluation substrate for a continuous SDR-vs-difficulty sweep, with its own source-file-level leakage-safe split — the triplet/leak-group split in (2) provably collapses to a single connected component once every heart file is combinatorially paired with every lung file, so the split unit for this substrate had to move from the mixture row up to the individual recording; and (6) six separation baselines run end-to-end through the harness — fixed bandpass filtering, a leakage-safe reproduction of a published supervised-NMF method, an unsupervised per-mixture NMF ablation, multi-stage Singular Spectrum Analysis (MSSA), a reproduction of a published EVMD-based method, and Conv-TasNet-lite, the first neural, end-to-end trained baseline. We report the audio-quality characteristics of the dataset, the structure of the recording-reuse and mixture-validity problems the split and the additive-triplet audit were built to solve, and all six baselines' 5-fold SDR/SIR/SAR results on the full native row set, the 36-row native-valid subset, and the 1500-row synthetic substrate, and situate this work against six related papers reviewed, none of which combine separation-quality reporting, isolated-vs. separated classification comparison, and a leakage-safe split on this dataset.

Building on that separation-quality pipeline, this revision adds (7) a downstream heart-sound classifier (MFCC features, an SVM), evaluated on the identical leak-group folds, with a class-grouping decision that reproduces Yaqub et al.'s own four-class scheme so results are directly comparable to their motivating 89%→41% result; (8) a controlled degradation scheme that interpolates continuously between a separation method's real output and the isolated ground truth, letting classification accuracy be measured as a continuous function of separation quality (SDR) rather than at two fixed endpoints, together with the generated sweep this produces across all six baselines; (9) a weight-sharing protocol (“Condition B”) for classifying separated audio with the exact same trained classifier used on isolated audio, and the resulting accuracy-vs-SDR curve and knee-point definition — the SDR below which separation stops beating not separating at all; (10) a second, architecturally distinct classifier (a log-Mel spectrogram + CNN) used as a robustness check on whether a knee point is a property of separation quality or an artifact of one architecture; and (11) a computational-cost analysis (multiply-accumulate operation counts

and measured desktop latency under a uniform protocol) placing every baseline’s separation quality against its compute cost on a Pareto plane. The six-baseline sweep, the resulting knee-point curve and the second architecture’s own curve are now complete and reported in full (Sections 8.7–8.9); an earlier revision reported them as partial results, and the oracle-mask ceiling check that revision listed as informal is now a committed, re-runnable script (Section 12).

The dataset’s provenance was verified independently after this pipeline was built (Section 3.1): the working copy is byte-identical to the Mendeley release, so the 4000 Hz sample rate and the 109/145 mixture rows that fail the named-reference additivity test are properties of HLS-CMDS as published rather than of a down-sampled redistribution.

Contents

1	Introduction	3
2	Background and Preliminaries	4
2.1	The problem: blind source separation	4
2.2	How do you know if a separation worked? BSS Eval (SDR/SIR/SAR)	5
2.3	Matrix factorization for audio: Non-negative Matrix Factorization (NMF)	6
2.4	An alternative decomposition: Singular Spectrum Analysis (SSA)	7
2.5	Optimization-based decomposition: Variational Mode Decomposition (VMD)	7
2.6	Deep learning for separation: convolutions, dilation, and Conv-TasNet	7
2.7	Evaluating machine learning fairly: cross-validation and data leakage	8
2.8	Classification metrics: accuracy, macro-F1, and the confusion matrix	9
2.9	Computational cost: MACs, latency, and the Pareto front	9
3	Dataset	10
3.1	Dataset provenance: verified against the release	10
3.2	Audio-quality audit	11
3.2.1	Per-class and per-location breakdown	11
4	Recording Reuse and the Leakage Problem	13
4.1	Content-hash audit	13
4.2	Leak groups	13
5	Mixture Validity: the Additive-Triplet Audit	14
6	Methodology	15
6.1	BSS Eval metrics	15
6.1.1	Validation	16
6.2	Evaluation harness	17
6.3	Synthetic mixture set and its own leakage-safe split	17
6.4	Baseline 1: bandpass filtering	18
6.5	Baseline 2: supervised NMF	18
6.6	Baseline 3: standard NMF (dictionary ablation)	19
6.7	Baseline 4: multi-stage Singular Spectrum Analysis (MSSA)	19
6.8	Baseline 5: EVMD (Enhanced Variational Mode Decomposition)	19
6.9	Baseline 6: Conv-TasNet-lite	20

7	Results	20
7.1	Full 145-row set	21
7.2	36-row additive-valid subset	21
7.3	Six methods, three substrates	22
8	Downstream Classification: Does Separation Hurt Accuracy?	24
8.1	Why one classifier architecture, deliberately, at first	24
8.2	The class-grouping decision	24
8.3	Architecture 1: MFCC summary statistics + an RBF-kernel SVM	25
8.4	Failure-mode analysis: the confusion matrix	25
8.5	A controlled way to sweep separation quality: the degradation scheme	26
8.6	Generating the SDR sweep	27
8.7	Condition B: classifying separated audio, and the weight-sharing decision	28
8.8	The accuracy-vs-SDR curve and the knee point	30
8.9	Robustness check: does a second, different classifier see the same knee?	32
9	Computational Cost	36
9.1	MACs per method	36
9.2	Desktop latency under a uniform protocol	37
9.3	The SDR-vs-compute plane	39
10	Embedded Deployment: Baseline 1 on a Microcontroller	40
10.1	The one intentional divergence: causal filtering	40
10.2	Verifying a port that has no reference implementation on the target	41
10.3	Results on silicon	41
10.4	A plausible measurement that was wrong by 18×	42
10.5	A tolerance that was calibrated on the wrong signal	43
10.6	Server against microcontroller, on identical work	43
10.7	Energy, where the answer is not yet known	45
10.8	Why the other five baselines do not run here	46
11	Related Work	47
11.1	Yaqub et al.: the motivating result, not an unrelated classification paper	48
11.2	Metrics caveats from the BSS Eval literature	49
12	Discussion	49
13	Limitations and Future Work	51

1 Introduction

Heart and lung sounds are conventionally recorded and classified in isolation, but a clinical stethoscope in practice picks up both simultaneously. HLS-CMDS [1] is, to our knowledge, the first published dataset to offer matched isolated and mixed cardiorespiratory recordings from the same underlying sources, which makes it possible to ask a question the isolated-only or mixed-only datasets used elsewhere in the literature cannot answer directly: *how much does separating heart from lung sound, before classifying it as normal or abnormal, cost in accuracy relative to classifying the isolated ground truth?*

That question is not rhetorical. Yaqub et al. [6] measured exactly this cost on this project’s own dataset and found it severe: a PCG classifier retrained on HLS-CMDS reaches 89.0% accuracy on clean, isolated recordings, but collapses to 41.0% when evaluated on heart sounds computationally separated from HLS-CMDS’s mixed recordings with a standard bandpass filter (their Experiment 4; see Section 11.1). That 89%→41% gap is the motivating result for this whole project: this report’s separation-quality metrics (SDR/SIR/SAR, Section 6.1) are what let that gap be explained mechanistically and traced out continuously against separation quality, rather than measured at only the two endpoints Yaqub et al. report.

Section 2 gives a from-scratch, student-level introduction to every technical idea the rest of this report depends on (blind source separation, BSS Eval, NMF, SSA, VMD, Conv-TasNet’s convolutional architecture, cross-validation and data leakage, classification metrics, and compute-cost measurement) so later sections can explain *why* a specific choice was made without re-deriving the underlying concept every time. Section 3 describes the dataset, the audio-quality audit run over it, and a provenance problem discovered after the pipeline below was built. Section 4 presents the recording-reuse finding that motivates a non-trivial train/evaluation split. Section 5 presents a second, independent data problem: most mixture rows turn out not to be acoustically related to their named sources at all. Section 6 describes the software components built to support this line of research: the split, the separation-quality metrics and their validation, the cross-validation harness, and all six separation baselines. Section 7 reports results on the full row set, the 36-row valid subset, and the 1500-row synthetic substrate. Section 8 builds on that separation-quality pipeline with a downstream classifier, the controlled degradation scheme and SDR sweep this project’s own C2 research question needs, the resulting accuracy-vs-SDR curve and knee point, and a second-architecture robustness check on that knee point. Section 9 measures every method’s computational cost (MACs and desktop latency) and places separation quality against compute cost on one plane. Section 11 surveys related work. Section 12 discusses the results and their implications, and Section 13 summarizes open work.

All software described here lives in a single repository, organized as: `src/load_dataset.py` (loading/validation, including the additive-triplet audit of Section 5), `src/split.py` (leakage-safe folds), `src/metrics.py` (BSS Eval), `src/eval_harness.py` (cross-validation), `src/baselines.py` (both separation baselines), `src/statistics/audio_quality.py` (dataset audit), and `src/visualization/` (plotting scripts), with `src/test_metrics.py`, `src/test_split.py`, and `src/test_load_dataset.py` as the accompanying test suites (31 tests total, all passing at the time of writing).

2 Background and Preliminaries

This section is written for a reader who knows general programming and basic statistics but has not necessarily studied audio signal processing, matrix factorization, or machine learning evaluation methodology before. Every later section builds on one of the ideas explained here, and refers back to this section by name rather than re-deriving the idea inline. If a later section says “because SAR captures this” or “this would be data leakage,” the explanation of *why* that matters is here, not repeated each time.

2.1 The problem: blind source separation

A stethoscope placed on a chest picks up heart sounds and lung sounds at the same time, physically summed into one waveform by the air and tissue they travel through before reaching the microphone. Call the true heart waveform $h(t)$, the true lung waveform $l(t)$, and the recorded mixture $m(t)$.

The simplest physical model of what happened is a linear mix:

$$m(t) = a \cdot (h(t) + l(t)) + n(t), \quad (1)$$

where a is an overall recording gain and $n(t)$ is any additional noise. *Source separation* means recovering estimates $\hat{h}(t)$ and $\hat{l}(t)$ from $m(t)$ alone. It is called *blind* when the separation method is not told what h and l individually sound like for this specific recording — it only gets $m(t)$, plus at most some general knowledge about what heart sounds and lung sounds tend to look like (learned from *other*, separate recordings).

This is a fundamentally underdetermined problem: Equation 1 is one equation with two unknown functions on the right-hand side, so there are infinitely many pairs $(\hat{h}, \hat{l})$ that add up to exactly the observed m . The only reason separation is possible at all is that heart and lung sounds are not arbitrary — they differ systematically in *where their energy sits*, whether that is described in frequency (Sections 6.4, 6.7, 6.8), in a learned spectral “dictionary” (Sections 6.5, 6.6), or in patterns a neural network discovers on its own from examples (Section 6.9). Every separation method in this report is, underneath its specific mathematics, a different way of exploiting that systematic difference to break the underdetermined problem.

2.2 How do you know if a separation worked? BSS Eval (SDR/SIR/SAR)

Once a method produces an estimate $\hat{s}_j$ for source j (heart or lung), the natural first idea is to compare it to the true source s_j with something like mean-squared error. That turns out to be a poor measure for audio, because it does not distinguish *why* $\hat{s}_j$ differs from s_j — a barely-audible time shift and a completely wrong signal can produce the same squared error. The BSS Eval framework of Vincent et al. [2] (Section 11.2) fixes this by decomposing the error into three parts with different *causes*:

$$\hat{s}_j = s_{\text{target}} + e_{\text{interf}} + e_{\text{artif}}, \quad (2)$$

where s_{target} is the best time-invariant-filtered version of the *true* source s_j that could be extracted from the estimate (so a harmless gain change or a short, fixed delay does not get punished), e_{interf} is the part of the estimate’s error that is explained by *other sources leaking in* (e.g. lung energy showing up in the heart estimate), and e_{artif} is whatever is left over — distortion the separation algorithm itself introduced that cannot be explained by leftover interference at all (filter ringing, NMF reconstruction error, neural-network hallucination). Three ratios follow directly, each answering a different question, always expressed in decibels (a base-10 logarithmic scale, $10 \log_{10}(\cdot)$, chosen because audio energy ratios span many orders of magnitude and human loudness perception is itself roughly logarithmic — a linear ratio would make large, easy-to-separate cases and small, hard cases look deceptively similar on a plot):

$$\text{SDR} = 10 \log_{10} \frac{\|s_{\text{target}}\|^2}{\|e_{\text{interf}} + e_{\text{artif}}\|^2} \quad \text{“how good is the estimate, overall?”} \quad (3)$$

$$\text{SIR} = 10 \log_{10} \frac{\|s_{\text{target}}\|^2}{\|e_{\text{interf}}\|^2} \quad \text{“how much of the other source leaked in?”} \quad (4)$$

$$\text{SAR} = 10 \log_{10} \frac{\|s_{\text{target}} + e_{\text{interf}}\|^2}{\|e_{\text{artif}}\|^2} \quad \text{“how much did the method itself distort the signal?”} \quad (5)$$

Why report all three instead of just SDR. A method can score well on one and poorly on another, and the gap is itself diagnostic. An aggressive bandpass filter, for instance, blocks the other source effectively (good SIR) but also rings and destroys the target’s own harmonics in the

process (poor SAR); a method that barely does anything scores the opposite way. SDR folds both effects into one number, which makes it convenient but also means a good SDR can hide a bad SAR that a downstream classifier might actually care about (Section 11.2’s point from Vincent et al. that a classifier can stay accurate despite real artifacts, or fail because of them, depending on whether the artifact crosses a decision boundary) — reporting only SDR would erase exactly the kind of trade-off Section 7 needs to describe.

2.3 Matrix factorization for audio: Non-negative Matrix Factorization (NMF)

Baselines 2 and 3 (Sections 6.5–6.6) work not on the raw waveform but on its *spectrogram*: split the waveform into short overlapping windows (a Short-Time Fourier Transform, STFT), take the magnitude of each window’s frequency spectrum, and stack these columns side by side into a matrix V of shape (frequency bins) $\times$ (time frames). Every entry of V is a non-negative energy value — there is no such thing as negative sound energy at a given frequency and time.

Non-negative Matrix Factorization asks: can V (size $F \times T$) be approximated as the product of two much smaller non-negative matrices, $V \approx WH$, where W (size $F \times K$) holds K “basis spectra” (recurring frequency shapes) and H (size $K \times T$) holds, for every time frame, how strongly each basis spectrum is active? K is chosen much smaller than F or T , so this is a compression that forces the algorithm to explain the whole spectrogram using only K reusable building blocks. The non-negativity constraint is what makes the result interpretable as a *parts-based* representation (unlike, say, PCA/SVD factorizations, which allow negative combinations and produce components that only make sense added and subtracted together, not individually): every basis spectrum in W is something a real sound could actually look like, and every column of H says “how much of each real sound is present right now,” non-negatively.

Fitting W and H is done by *multiplicative updates* (Lee & Seung, 2001): starting from a random non-negative guess, W and H are repeatedly rescaled by a ratio that is guaranteed to be non-negative and that provably decreases the reconstruction error (specifically the KL-divergence between V and WH used here) at every step. This is a form of coordinate descent — it is simple to implement and, unlike generic gradient descent, never needs a learning rate or risks producing a negative value that would have to be clipped.

Supervised NMF (Baseline 2, Section 6.5) exploits this by learning W *separately* for heart and lung, from clean, isolated recordings of each, before ever looking at a mixture — exactly analogous to learning a small palette of “basis colors” from pure paint samples. The two W matrices are then frozen and stacked together, and only H is re-solved (with the same multiplicative-update rule, holding W fixed) against a specific held-out *mixture*’s spectrogram. This answers “how much of each already-known basis spectrum would need to be present, at each moment, to reconstruct this new, unlabeled mixture?” — the same idea as decomposing an unknown mixed color as a combination of known primary colors. Once H says how much of the heart-dictionary and lung-dictionary explain the mixture at each time-frequency point, that proportion becomes a *soft mask*: the fraction of the mixture’s complex spectrogram at each point assigned to heart is $W_{\text{heart}}H_{\text{heart}}/(W_{\text{heart}}H_{\text{heart}} + W_{\text{lung}}H_{\text{lung}})$, applied directly to the mixture’s own (complex-valued) STFT before inverting back to a waveform — this is a *Wiener-style* mask, the standard way to turn an energy decomposition into an actual audio estimate. **Standard NMF** (Baseline 3, Section 6.6) is the same machinery with one piece removed: no pre-learned dictionary at all, both W and H are factorized fresh out of each mixture’s own spectrogram (a purely *blind*, or *transductive*, use of NMF), which isolates how much of Baseline 2’s performance is actually attributable to its learned, frozen dictionary rather than to NMF factorization itself.

2.4 An alternative decomposition: Singular Spectrum Analysis (SSA)

Baseline 4 (Section 6.7) uses a completely different mathematical tool: Singular Spectrum Analysis. The construction has three steps. First, *embedding*: turn the 1-D waveform of length N into a 2-D matrix by sliding a window of length L along it one sample at a time and stacking each window as a column, giving an $L \times (N - L + 1)$ *trajectory matrix*. This turns “what happens over time” into “what does this matrix’s structure look like,” which linear algebra has strong tools for. Second, *decomposition*: apply the Singular Value Decomposition (SVD), which writes any matrix as a sum of rank-1 pieces (each an outer product of one “time pattern” vector and one “lag pattern” vector) ordered by how much of the trajectory matrix’s total energy each piece explains. These rank-1 pieces are the same mathematical object behind PCA, and each one corresponds to some characteristic oscillation frequency in the original signal — SSA does not force this frequency interpretation, but it falls out naturally because a matrix built from a windowed periodic signal has exactly this kind of low-rank structure. Third, *diagonal averaging* (Hankelization): each selected rank-1 piece, still shaped like an $L \times (N - L + 1)$ matrix, is converted back into a 1-D *reconstructed component* (RC) by averaging along its anti-diagonals, undoing the windowing from step one.

Given a set of RCs, each with its own dominant frequency, separating heart from lung reduces to sorting them by that frequency and grouping the low-frequency ones into a cardiac output and the higher-frequency ones into a respiratory output — which is exactly what MSSA’s two-stage design does (a first pass with a fixed cardiac/respiratory frequency threshold, then a second SSA pass on the leftover residual to more carefully pick out the respiratory content by how much variance each RC explains and how correlated it is with what has already been selected). No dictionary is learned from other recordings anywhere in this process — SSA operates entirely on the one signal it is given, which is why Baseline 4 (like Baseline 1) is a zero-training method with no leakage exposure at all (Section 4).

2.5 Optimization-based decomposition: Variational Mode Decomposition (VMD)

Baseline 5 (Section 6.8) uses Variational Mode Decomposition (Dragomiretskiy & Zosso, 2014), which frames separation as an explicit optimization problem rather than a matrix factorization or a fixed filter: find K “modes,” each a signal that is as narrow-band as possible (concentrated tightly around its own, simultaneously-discovered center frequency ω_k), subject to the constraint that all K modes sum back to the original signal exactly. “As narrow-band as possible” is made precise by minimizing the modes’ total bandwidth (measured via the signal’s Hilbert transform, which shifts a real signal’s spectrum so its energy content at each frequency can be isolated and penalized). This joint problem — K unknown signals *and* K unknown center frequencies, all coupled through one shared constraint — is solved by the Alternating Direction Method of Multipliers (ADMM), a general optimization strategy for exactly this situation: instead of solving for everything at once, alternate between updating each mode (holding all others and the frequencies fixed) and updating each center frequency (holding the modes fixed), repeating until the modes stop changing. Once the mode with the lowest center frequency is identified as the cardiac component, subtracting it from the mixture gives the respiratory estimate. K itself is not known in advance and is swept from 2 upward until a paper-specified quality check passes (or a ceiling is hit) — a further practical detail documented in Section 6.8.

2.6 Deep learning for separation: convolutions, dilation, and Conv-TasNet

Baseline 6 (Section 6.9) replaces a hand-designed transform (STFT, trajectory matrix, mode decomposition) with weights learned end-to-end from data. Its core operation is the 1-D convolution:

a short, learned filter (a small vector of weights) is slid across the waveform, and at each position it computes a weighted sum of the samples underneath it — conceptually the same sliding-window idea as Baseline 1’s Butterworth filter (Section 6.4), except the filter’s coefficients are *learned* from examples rather than hand-derived from a PSD survey. Stacking many such convolutional layers, each one operating on the previous layer’s output, lets the network build up increasingly abstract, longer-range patterns from short, local ones.

A plain stack of convolutions would need very many layers to “see” a useful amount of context (each layer only extends what the next one can see by roughly its own filter length), which is where *dilated* convolutions come in: a dilated filter skips input positions at a fixed spacing, so the same small number of weights covers a much wider span of input without adding parameters. Stacking layers with exponentially increasing dilation (as Conv-TasNet’s Temporal Convolutional Network, TCN, does: dilation $2^0, 2^1, 2^2, \dots$) lets the network’s effective *receptive field* — how much of the input any one output value depends on — grow exponentially with depth instead of linearly, which is how Baseline 6 covers roughly 1.27 s of audio context with only a handful of layers (Section 6.9).

Conv-TasNet’s overall architecture has three pieces trained jointly: an *encoder* (a bank of learned 1-D convolution filters that plays the same role the STFT plays for the NMF baselines, except the filter shapes are learned rather than fixed sinusoids), a *separator* (the dilated TCN described above, which looks at the encoded mixture and predicts one soft mask per source), and a *decoder* (the mathematical inverse of the encoder, turning a masked encoded representation back into a waveform). Training this end-to-end means the network never has to be told what a “basis spectrum” or a “frequency band” is — it discovers whatever representation and masking strategy minimizes its training loss directly, which is both Baseline 6’s main advantage (it can in principle discover structure the other five methods’ fixed transforms cannot represent) and its main practical cost (it needs enough labeled training examples to find that structure reliably, which Section 6.9 and Section 7 both flag as a real limitation on this dataset’s small recording pool).

2.7 Evaluating machine learning fairly: cross-validation and data leakage

Any time a method is *fit* to data (a learned dictionary, a trained network, even a hand-tuned filter cutoff), evaluating it on the same data it was fit on overstates how well it will do on genuinely new recordings — the method may simply have memorized quirks of that specific data rather than learned something general. The standard fix is to hold out some data, fit only on the rest, and evaluate only on the held-out part. *k-fold cross-validation* extends this so that every data point gets to be in the held-out set exactly once (across k different “folds”), so the whole dataset is used for both fitting and evaluation without ever evaluating a fold’s result using data it was fit on — important on a dataset this small, where reserving a large, fixed holdout would leave too little data to fit anything meaningful.

Data leakage is the general name for evaluating a model in a way that lets information from the held-out set influence what it was fit on, even indirectly, which silently inflates the measured score without the model actually generalizing any better. The textbook example is fitting and evaluating on data that turn out to contain duplicate rows: if a near-copy of a test example sits in the training set, the model does not have to generalize to score well on it. This project’s leak-group split (Section 4) exists precisely because HLS-CMDS has exactly this problem in a non-obvious form: the same physical recording appears multiple times under *different ID strings* (once standalone in `HS.csv/LS.csv`, and again inside one or more `Mix.csv` rows), so a standard, generic k -fold split — which only knows about row IDs, not about which underlying audio content those IDs point to — would silently leak a mixture’s own ground truth into the pool used to fit its separation dictionary. Detecting this required hashing raw audio content directly rather than trusting the ID columns

(Section 4), and the resulting split unit is not the row but a *leak group* (a connected component of rows that share audio) — the minimal change to standard cross-validation that actually closes this specific leak.

2.8 Classification metrics: accuracy, macro-F1, and the confusion matrix

Section 8 onward evaluates a *classifier* (does a heart recording belong to Normal, Murmur, Extra Sound, or Rhythm Disorder?), which needs its own vocabulary. *Accuracy* is simply the fraction of examples classified correctly, but it has a well-known blind spot on imbalanced data: if 24 of 50 recordings are Murmur and a classifier always predicts “Murmur” regardless of the input, it scores 48% accuracy while having learned nothing useful about the other three classes. *Macro-F1* is one standard fix: compute the F1 score (the harmonic mean of precision and recall) separately for *each* class, then average those per-class scores with equal weight — so a model that nails the common class but fails every rare one no longer gets to hide behind the common class’s size. A *confusion matrix* is the raw material both metrics are computed from: a table whose rows are the true class and columns are the predicted class, so cell (i, j) counts how many examples of true class i were predicted as class j . The diagonal is what a classifier got right; everything off the diagonal is a specific, nameable mistake (“Rhythm Disorder recordings are most often predicted as Murmur”), which is far more actionable than a single aggregate number and is exactly the level of detail Section 8.4 uses to describe each architecture’s specific failure pattern.

2.9 Computational cost: MACs, latency, and the Pareto front

Section 9 asks a different kind of question: not “how good is this method,” but “how expensive is it to run.” The atomic unit of cost used throughout that section is the *multiply-accumulate* (MAC): one multiplication followed by one addition into a running total, which is the single operation almost every method in this report is built out of at its core, whether it is a matrix multiplication inside NMF, a convolution inside Conv-TasNet, or an SVD inside SSA. Counting MACs gives a hardware-independent proxy for “how much arithmetic work” a method requires — useful because it can be computed analytically from a method’s own equations, without needing to actually run it. It is, however, only a *proxy*: how quickly a given MAC count actually executes on real hardware depends heavily on how well that arithmetic maps onto the hardware’s own optimized code paths (vectorized instructions, cache-friendly memory access, a mature compiled library vs. a direct, unoptimized implementation of a paper’s own pseudocode). Section 9 measures both MACs (the theoretical proxy) and real wall-clock latency (the measured outcome) *separately*, specifically because they turn out to disagree for at least one method here — a divergence that a MACs count alone could never reveal.

Comparing several methods on two axes at once (quality and cost) uses the idea of a *Pareto front*, borrowed from multi-objective optimization: method A is said to *dominate* method B if A is at least as good as B on every axis being compared and strictly better on at least one. A dominated method is never the right choice, no matter how the two axes are weighted against each other, because some other method is simply better in every way that matters; a method that is *not* dominated by anything represents a genuine trade-off (better quality at higher cost, or lower cost at some quality loss) that a deployment decision would actually have to weigh. Section 9.3 applies this directly to the six separation baselines’ SDR-vs-compute trade-off.

3 Dataset

HLS-CMDS [1] records heart and lung sounds from a CAE Juno mid-fidelity patient simulator with swappable male/female chest skins, captured with a digital stethoscope in a quiet room. Heart sounds are recorded at six chest landmarks (Apex, RUSB, LUSB, LLSB, RC, LC) across ten sound-type classes (Normal, Late Diastolic Murmur, Mid Systolic Murmur, Late Systolic Murmur, Atrial Fibrillation, S4, Early Systolic Murmur, S3, Tachycardia, AV Block); lung sounds are recorded at six anterior landmarks (LLA, LMA, LUA, RLA, RMA, RUA) across six classes (Normal, Wheezing, Fine Crackles, Rhonchi, Pleural Rub, Coarse Crackles). The dataset ships three CSV manifests: `HS.csv` (50 isolated heart recordings), `LS.csv` (50 isolated lung recordings), and `Mix.csv` (145 mixed recordings, each row also naming the heart and lung recording used to build that mixture).

3.1 Dataset provenance: verified against the release

The dataset copy every result in this report is computed on is the released one. The working copy in this project (HLS-CMDS/) was obtained from github.com/Torabiy/HLS-CMDS rather than the Mendeley release (data.mendeley.com/datasets/8972jxbpmp/3) the project charter specifies, and an internal review (*TEEP2026_Sprint0_Review*, 2026-08-18) flagged the resulting possibility that everything here had been measured on a downsampled convenience mirror. That possibility was tested directly and did not hold: a fresh Mendeley download (v3, retrieved 2026-08-19), checksummed independently of this project’s own tooling, is byte-identical to the working copy on all six top-level artifacts (Table 1).

Table 1: Provenance check. Mendeley v3, retrieved 2026-08-19; MD5 over each top-level artifact, identical between the fresh download and the working copy.

Artifact	MD5
HS.csv	b61a866d2e8d70ae2910e4c049327d1e
HS.zip	7d678aff168184c13d8893b3bef4f4ba
LS.csv	fa40494d1dcbcf173ee5e9583b3faf82
LS.zip	d56a0d8ce5941342f33bfa9b04ef2bd9
Mix.csv	377b55ea6361ff7c48994cddaf5c6e6f
Mix.zip	f28243aa5394324c58f90a466d60c2f4

Two findings survive that check as properties of the published dataset, which is why this subsection is retained rather than deleted:

- **Sample rate.** Every file is 4000 Hz (Section 3.2), not the 22,050 Hz the HLS-CMDS descriptor paper states — a $5.5\times$ mismatch belonging to the release itself. Whether the descriptor misstates the rate or the figure was mis-transcribed downstream of it is not resolved here. It is not a problem for the science, since heart energy sits at 20–200 Hz and lung at 100–1000 Hz (Section 6.4), comfortably under a 2000 Hz Nyquist; it is a problem for tooling, and Section 6.9 documents the consequence.
- **Mixture validity.** 109 of 145 `Mix.csv` rows’ *mixed* recording is acoustically unrelated to its named heart and lung sources (Section 5); only 36 rows are usable as paired separation ground truth. Re-downloading cannot recover them, because the re-download is the same bytes.

Both figures come from HS/LS/Mix.csv’s own audio content, not from metadata, so they are not artifacts of a mislabeled manifest. Every separation-quality number in this report is accordingly a measurement on HLS-CMDS as published.

3.2 Audio-quality audit

`src/statistics/audio_quality.py` scans every `.wav` file referenced by the three manifests (535 files total: $50 + 50 + 145 \times 3$, since each mix row’s heart, lung, and mixed components are separate files) and reports duration, sample rate, channel count, and clipping (samples pinned at the 16-bit PCM full-scale rail, ± 32767), aggregated per sound-type class and per recording location.

Format. Every file is 4000 Hz, mono, exactly 15.0 s (60,000 samples) — perfectly uniform, no resampling or trimming required before feature extraction. This uniformity is itself part of the provenance discrepancy: 4000 Hz is not the sample rate the HLS-CMDS descriptor paper documents (Section 3.1).

Clipping. 36 of 535 files (6.7%) clip, but only by a single sample each ($1/60,000 \approx 0.0017\%$ of the file), and exclusively in the *mixed* component of `Mix.csv` rows (never in `HS.csv`, `LS.csv`, or the mix set’s own heart/lung components) — specifically row M0087 and the contiguous run M0111–M0145. This set of 36 rows is exactly the set Section 5’s additive-triplet audit independently finds acoustically valid, which resolves the mechanism: these are the rows where *mixed* was actually built by summing the heart and lung waveforms and then peak-normalizing the result (an observed gain of 3.5–135 $\times$ across the 36 rows), and peak normalization of an already-full-scale sum pins one sample at the 16-bit rail. It is a normalization fingerprint, not a recording defect, and it has not been reported upstream.

3.2.1 Per-class and per-location breakdown

Beyond the dataset-wide summary above, `audio_quality.py` also aggregates duration, sample rate, and clipping *per class* and *per recording location*, over the 100 isolated recordings (`HS.csv` + `LS.csv`). Duration (15.0 s, std 0.0) and sample rate (4000 Hz) are constant across every class and every location — omitted from Tables 2–3 below since there is no variation to report — so the only columns that carry information are the class/location’s sample count and its clipped-file count.

Table 2: Per-class breakdown, isolated recordings only (HS.csv/LS.csv).

Component	Class	n	Clipped
Heart	Normal	9	0
Heart	Mid Systolic Murmur	7	0
Heart	Late Diastolic Murmur	6	0
Heart	Early Systolic Murmur	6	0
Heart	Late Systolic Murmur	5	0
Heart	S3	5	0
Heart	Atrial Fibrillation	4	0
Heart	AV Block	3	0
Heart	Tachycardia	3	0
Heart	S4	2	0
Lung	Normal	12	0
Lung	Coarse Crackles	9	0
Lung	Pleural Rub	9	0
Lung	Rhonchi	8	0
Lung	Wheezing	7	0
Lung	Fine Crackles	5	0

Table 3: Per-location breakdown, isolated recordings only (HS.csv/LS.csv).

Component	Location	n	Clipped
Heart	LUSB	13	0
Heart	Apex	10	0
Heart	LLSB	10	0
Heart	RUSB	7	0
Heart	LC	6	0
Heart	RC	4	0
Lung	LUA	11	0
Lung	RLA	10	0
Lung	LMA	9	0
Lung	LLA	8	0
Lung	RUA	7	0
Lung	RMA	5	0

Why this breakdown is usable. Three properties make it trustworthy as a basis for downstream decisions, rather than just a descriptive aside. First, it is measured directly from the audio files themselves (`wave` header + sample scan), not read off manifest metadata, so it cannot inherit a CSV labeling error the way a metadata-only summary could. Second, duration and sample rate being *exactly* constant (std 0.0) across every one of the 16 classes and 12 locations means no class or location requires different preprocessing (resampling, padding, trimming) before feature extraction — a per-class confound that would otherwise have to be ruled out separately for every downstream experiment. Third, the clipped-file count is zero for *every* class and *every* location here, which cross-checks the dataset-wide clipping finding (Section 3.2) at finer granularity: clipping is confined

entirely to `Mix.csv`’s *mixed* component and does not concentrate in any particular heart/lung class or recording site. That matters because clipping correlated with a specific class would risk a classifier learning the clipping artifact as a spurious class signal; here that risk is ruled out by direct measurement rather than assumed.

Why it is good news, with one caveat. The absence of any per-class or per-location quality defect means the planned classification stage (PROTOCOL.md §5.3) can treat every isolated recording as equally trustworthy input — there is no class whose recordings are disproportionately degraded and would need to be excluded or reweighted on quality grounds alone. The caveat is class *size*, not quality: several heart classes are very small (S4 $n=2$, AV Block and Tachycardia $n=3$ each), which Table 2 makes precise rather than leaving as an impression. Under Section 4’s leak-group-respecting 5-fold split, a class with only 2 recordings cannot appear in every fold’s training set, which will show up as unstable or undefined per-class accuracy for those classes in the classification stage — worth flagging now, before that stage is built, rather than discovering it as a confusing result later (Section 13).

4 Recording Reuse and the Leakage Problem

A naïve evaluation protocol would split `Mix.csv` by row (or by `Mixed Sound ID`) into train/test folds, and fit any separation dictionary on all of `HS.csv`/`LS.csv`. Both choices leak ground truth into training, and the reason is specific to how HLS-CMDS was assembled, not a generic caveat.

4.1 Content-hash audit

`HS.csv` and `Mix.csv` use *different ID namespaces* for the same underlying files (e.g. `HS.csv`’s `F_N_RC` vs. `Mix.csv`’s `H0001`), so file identity cannot be checked by ID string comparison. Instead, `src/split.py` hashes the raw PCM samples of every file (ignoring WAV headers, via 16-bit PCM read + MD5) to find:

- `Mix.csv`’s 145 rows draw from only **89** distinct heart recordings and **74** distinct lung recordings (by content), each reused across multiple mix rows — up to $6\times$ for a single heart recording.
- **32** of those 89 heart recordings, and **30** of those 74 lung recordings, are byte-identical (by PCM content) to a recording also listed standalone in `HS.csv`/`LS.csv` — the exact pool a separation dictionary would be fit on.

4.2 Leak groups

Because recordings are reused *across* mix rows independently for the heart and lung components, the leakage-safe unit is not a single mix row but a *connected component*: two rows are linked whenever they share a heart or a lung recording (directly or transitively through a third row). Union-find over all 145 rows on this relation yields **29** connected components (“leak groups”), ranging in size from 1 to 32 rows (Table 4).

Table 4: Leak-group size distribution over `Mix.csv`’s 145 rows (29 groups total; the sizes sum to 145). An earlier revision listed 13 groups of size one, which summed to 144 rows and 28 groups — a transcription error, corrected against `split.py`’s output.

Group size	32	22	19	13	11	9	4	3	2	1
# groups of this size	1	1	1	1	1	1	2	3	4	14

`split.py`’s `assign_folds()` partitions these 29 groups (never splitting one across folds) into k folds via a greedy largest-group-to-smallest-fold assignment, giving reasonably balanced folds despite the size-32 outlier (Table 5). `dictionary_pool()` then returns, for a given held-out fold, exactly the `HS.csv/LS.csv` rows whose content does *not* appear anywhere in that fold’s evaluation mixtures — the pool any dictionary-learning or model-fitting step must be restricted to when that fold is held out.

Table 5: 5-fold split (seed 0): row counts and resulting dictionary-fitting pool sizes.

Fold	Eval rows (of 145)	HS pool (of 50)	LS pool (of 50)
0	32	44	46
1	29	43	46
2	28	44	47
3	28	40	43
4	28	47	38

This split governs *both* dictionary/model fitting and evaluation — not evaluation alone — which is the property `src/test_split.py` verifies directly (Section 6.1.1).

5 Mixture Validity: the Additive-Triplet Audit

Section 4’s content-hash audit answers “does a mix row’s ground truth leak into the pool used to fit a separation model.” It says nothing about a logically prior question: is the *mixed* recording named in a `Mix.csv` row actually related to the heart and lung recordings named alongside it at all? `load_dataset.py`’s `verify_mix_triplets()` only checks that the three files exist, are uniquely paired, and agree on format (Section 3) — it would pass even if *mixed* were a completely unrelated recording bundled under the same row by a manifest error.

`verify_additive_triplets()` tests the assumption every separation baseline and BSS Eval score in this pipeline actually relies on: $\text{mixed} \approx a \cdot (\text{heart} + \text{lung})$ for some scalar gain a . For each row it fits the least-squares a minimizing $\|\text{mixed} - a(\text{heart} + \text{lung})\|^2$, then reports the relative residual $\|\text{mixed} - a \cdot \text{summed}\|/\|\text{mixed}\|$. This measure is empirically bimodal with a wide gap: genuinely additive rows land at residual $\sim 10^{-8}$ – 10^{-4} (limited by 16-bit quantization), acoustically unrelated rows land at residual ~ 1 (no shared energy at all), and no row falls in between — so the exact threshold (0.001) is not a sensitive tuning choice.

Result: only 36 of 145 rows (24.8%) pass. The other 109 rows’ *mixed* file shares essentially no energy with its own named heart and lung recordings. This independently reproduces *TEEP2026.Sprint0_Review*’s count on the same dataset copy, and (as noted in Section 3.2) the 36 valid rows are exactly the 36 rows showing the single-sample clipping fingerprint — two independent measurements (spectral/waveform residual, and a clipping artifact of the mixing process)

converging on the same row set is itself evidence the finding is a real property of the data, not a bug in either check.

This has two consequences for the rest of this report. First, it is a second, independent problem from leakage (Section 4): a row can be leakage-safe and still be acoustically meaningless, or vice versa. Second, because 75% of `Mix.csv` is affected, every result in Section 7 is reported *twice* — on the full 145-row set (for continuity with earlier work and Table 5’s split) and on the 36-row valid subset alone (Table 6) — since averaging rows that fail the additive-reference test into a headline separation-quality number would mix “the method failed to separate” with “there was nothing coherent to separate in the first place.”

Table 6: Additive-triplet audit result on HLS-CMDS as published (Mendeley v3, provenance verified in Section 3.1).

	Rows	% of 145
Additive (mixed $\approx a \cdot (\text{heart} + \text{lung})$)	36	24.8%
Not additive (acoustically unrelated)	109	75.2%

Sensitivity of the verdict to the strictness of the model. A reviewer’s natural objection is that a one-gain, zero-lag fit would reject a mixture for trivial reasons — a DC offset, an inverted polarity, a few samples of misalignment, a different gain on each source. Each was tried on all 145 rows (`additive_audit_sensitivity.py`): the mean removed before fitting; the gain allowed to be negative (the least-squares gain is already unconstrained, and 52 rows do choose a negative one — none of them passes at any threshold); the summed reference shifted by the cross-correlation-optimal integer lag within ± 100 samples; separate gains for heart and lung; and separate gains with separate per-source lags. Under every one of the seven models the count at 10^{-3} is 36, with no row gaining or losing the verdict. The passing cluster’s residuals span $0.9\text{--}4.5 \times 10^{-4}$ and the failing cluster’s $0.94\text{--}1.0$ even under the loosest model — a gap of more than $2000\times$ — so the thresholds 10^{-2} and 10^{-3} give the same 36 rows, while 10^{-4} admits only 3, because 16-bit quantization alone puts a $\sim 10^{-4}$ floor on any genuinely additive row. Among the 36, the optimal lag is 0 for every row; among the 109 the optimal lag is spread uniformly to the ± 100 -sample edge, which is what cross-correlating two unrelated signals produces. Every file reads 4000 Hz. The 109 rows are therefore not near-misses of the additive model under any relaxation tried; they are rows whose mixture shares no energy with the two recordings named alongside it.

6 Methodology

6.1 BSS Eval metrics

Separation quality is scored with the classic BSS Eval decomposition [2]: an estimated source $\hat{s}_j$ is decomposed into a target component (the best time-invariant-filtered version of the true source s_j , filter length up to 512 taps), an interference component (leakage from the other true sources), and an artifacts component (residual distortion not explained by either). From this decomposition:

$$\text{SDR} = 10 \log_{10} \frac{\|s_{\text{target}}\|^2}{\|e_{\text{interf}} + e_{\text{artif}}\|^2} \quad (\text{Signal-to-Distortion Ratio}) \quad (6)$$

$$\text{SIR} = 10 \log_{10} \frac{\|s_{\text{target}}\|^2}{\|e_{\text{interf}}\|^2} \quad (\text{Source-to-Interference Ratio}) \quad (7)$$

$$\text{SAR} = 10 \log_{10} \frac{\|s_{\text{target}} + e_{\text{interf}}\|^2}{\|e_{\text{artif}}\|^2} \quad (\text{Sources-to-Artifacts Ratio}) \quad (8)$$

`src/metrics.py` wraps `mir_eval.separation.bss_eval_sources` [3], specialized for this dataset’s 2-source (heart, lung) case:

```
def evaluate_heart_lung(heart_ref, lung_ref, heart_est, lung_est,
                       compute_permutation=True) -> dict:
    """Returns {"heart": {"sdr":..., "sir":..., "sar":...},
                "lung": {"sdr":..., "sir":..., "sar":...}}"""
```

together with `evaluate_mix_row()/evaluate_dataset()`, which load ground truth via `load_dataset.py` and run a caller-supplied `separate_fn(mixed, sr) → (heart_est, lung_est)` over one or all `Mix.csv` rows, and `summarize_by_class()` for per-class/per-location aggregation. `mir_eval` 0.8 deprecated `bss_eval_sources` in favor of an unpublished museval-style API; since no working replacement exists yet, the deprecated call is used with its `FutureWarning` suppressed locally.

6.1.1 Validation

`src/test_metrics.py` (10 tests) validates the wrapper two ways:

- **Cross-checks against `mir_eval` directly** — on synthetic arrays and on real dataset audio, the wrapper’s output is compared bit-for-bit against a raw `mir_eval` call on the same stacked arrays, catching wrapper bugs (stacking order, trimming, permutation labeling) independent of whether the underlying metric itself is correct.
- **Synthetic mixtures with closed-form ground truth.** White-noise sources are corrupted in ways whose correct BSS Eval score is analytically known, and the measured score must land within tolerance:

Table 7: Synthetic ground-truth validation (`test_metrics.py`).

Construction	Theoretical target	Tolerance	Result
$\hat{s} = s$ (identity)	$\text{SDR/SIR/SAR} \rightarrow \infty$	$> 100 \text{ dB}$	$> 300 \text{ dB}$
$\hat{s} = s + n$, SNR 20 dB	$\text{SDR} \approx \text{SAR} \approx 20 \text{ dB}$	$\pm 1.0 \text{ dB}$	within tolerance
$\hat{s} = s + n$, SNR 6 dB	$\text{SDR} \approx \text{SAR} \approx 6 \text{ dB}$	$\pm 1.0 \text{ dB}$	within tolerance
$\hat{s}_j = s_j + \alpha s_k$, $\alpha=0.1$	$\text{SIR} \approx -20 \log_{10} \alpha = 20 \text{ dB}$	$\pm 1.0 \text{ dB}$	within tolerance
$\hat{s}_j = s_j + \alpha s_k$, $\alpha=0.3$	$\text{SIR} \approx 10.46 \text{ dB}$	$\pm 1.0 \text{ dB}$	within tolerance

`src/test_split.py` (15 tests) validates the split, with the load-bearing test being `test_dictionary_pool_excludes_all_held_out_recordings`: for every fold, it hashes every recording in that fold’s dictionary pool and asserts *zero* overlap with any recording used in that fold’s held-out evaluation mixtures. It also checks leak groups never split across folds, fold coverage/balance, split reproducibility under a fixed seed, and that `cross_validate` (Section 6.2) actually hands each fold its

restricted pool rather than the unrestricted full dataset. All 25 tests across both files pass (~ 2 minutes total, dominated by running real BSS Eval over 15-second real recordings repeatedly).

6.2 Evaluation harness

`src/eval_harness.py` ties Sections 4 and 6.1 together:

```
def cross_validate(fit_and_separate_fn, n_folds=5, seed=0):
    """For each fold: fit_and_separate_fn(hs_allowed, ls_allowed) -> separate_fn,
    evaluate separate_fn on the held-out fold via metrics.evaluate_dataset.
    Returns (results_df, fold_summary, cv_summary)."""
```

`fit_and_separate_fn` is the caller’s separation method or model; the harness supplies each fold’s leakage-restricted dictionary pool and handles fold iteration, per-row scoring, and aggregation. An optional `mix_df` argument restricts evaluation to a caller-supplied subset of `Mix.csv` (folds are then computed within that subset only) — used in Section 7 to run every baseline on Section 5’s 36-row valid subset without duplicating the fold-assignment logic. Results are aggregated three ways: mean SDR/SIR/SAR per (fold, source); mean $\pm$ std *across* folds per source (`aggregate_across_folds`, fold-level dispersion); and mean, median, and std pooled directly over every evaluated row per source (`summarize_pooled`, row-level dispersion). The last of these was added after an observation in *TEEP2026-Sprint0-Review*: averaging within folds before taking a standard deviation can understate the true row-to-row spread substantially (an across-fold figure of 6.46 ± 3.71 dB for the same data pools to 6.30 ± 39.38 dB row-by-row in one comparison the review made), so the two are not interchangeable and a report should say which one it means. Both are reported in Section 7.

6.3 Synthetic mixture set and its own leakage-safe split

Only 36 of `Mix.csv`’s 145 rows are additive (Section 5), too few to source a continuous SDR-vs-difficulty sweep. `synthetic_mix.py` builds a combinatorial synthetic set instead: every one of the 50 heart recordings paired with every one of the 50 lung recordings, mixed as $\text{mixed} = a \cdot (\text{heart} + \text{lung}) + \text{noise}$ — the same functional form `verify_additive_triplets()` (Section 5) already shows the native additive rows follow, not an arbitrary synthetic recipe. The gain a is drawn log-uniformly per source pair from the range empirically fitted on the 36 native additive rows’ own gains; additive Gaussian noise is scaled to a target SNR swept over $\text{SNR_SWEEP_DB} = \{-5, 15, 35\}$ dB (trimmed from 5 levels after measuring that a 5-level sweep across all baselines would run multi-hour) to give a controllable difficulty axis, rather than Han & Quan’s own fixed 2%-RMS noise level. `validate_against_native` confirms the construction is faithful, not just claimed to be: rebuilding each of the 36 native additive rows from their own fitted gain and zero noise, then re-running `verify_additive_triplets()` against the synthetic result, reproduces all 36 rows at the same $\sim 10^{-4}$ residual the genuine native rows occupy.

The split has to move from mixture row to source file. Section 4’s triplet-level split assigns *leak groups* — connected components of mix rows sharing a heart or lung recording — to folds, which is the right unit when each recording appears in only a few of the 145 native mixtures. Full combinatorial pairing breaks that assumption: one heart file now appears in 50 mixtures, so within a few hops every heart file transitively connects to every lung file through some shared mixture, collapsing the entire synthetic set into a single connected component with no split left to take. `assign_source_folds` resolves this by assigning each of the 50 heart files and 50 lung files *independently* to one of 5 folds; a synthetic pair (h, l) is only used for held-out evaluation under fold k when $\text{fold}(h) = \text{fold}(l) = k$, and `dictionary_pool_synthetic` excludes *both* sources

of every held-out pair from that fold’s dictionary-fitting pool — the same no-overlap contract as `split.dictionary_pool` (Section 4), re-derived at the file level rather than the leak-group level. This is not a cosmetic change: getting it wrong (e.g. falling back to a row-level or leak-group split that no longer separates anything) would let a fold’s dictionary see a file that also appears in that fold’s held-out evaluation pairs, inflating every downstream SDR/SIR/SAR number without any test failing to flag it. `test.synthetic_mix.py` makes this a checked invariant rather than a one-off claim: for every fold, `test.dictionary_pool_excludes_both_sources_of_held_out_pairs` asserts zero overlap between that fold’s dictionary pool and either source recording of any pair held out in that fold, alongside `test.only_same_fold_pairs_are_included` confirming every synthetic row actually satisfies $\text{fold}(h)=\text{fold}(l)$.

6.4 Baseline 1: bandpass filtering

A fixed, non-adaptive separation method serves as the sanity floor any learned method should comfortably beat — if it doesn’t, that signals a bug in the learned method rather than a genuinely hard problem. Cutoffs were derived from this dataset’s own spectral content rather than assumed from generic audio literature: Welch’s method (`scipy.signal.welch`, 1024-sample segments) was run over every isolated recording in `HS.csv` and `LS.csv` separately and averaged per source, showing heart energy dominant below ~ 150 – 200 Hz, lung energy dominant from ~ 200 Hz up to ~ 700 – 800 Hz, and both sources falling into the recording noise floor above ~ 1 kHz (Table 8 gives representative values; no PSD plot is included in this draft).

Table 8: Representative averaged, peak-normalized PSD values (dB) from the empirical audit.

Frequency	Heart (dB)	Lung (dB)	Dominant source
93.8 Hz	−2.3	−12.1	Heart
187.5 Hz	−15.7	−24.6	Heart (crossover region)
500 Hz	−26.7	−14.4	Lung
1000 Hz	−50.4	−50.2	Neither (noise floor)

Based on this, Baseline 1 uses a heart band of 20–200 Hz and a lung band of 150–1000 Hz, applied as a 4th-order zero-phase Butterworth bandpass (`scipy.signal.butter` in second-order-section form, filtered with `sosfiltfilt` to avoid the group-delay a single-pass IIR filter would introduce). The 150–200 Hz overlap between the two bands is intentional: the PSD audit shows both sources carry genuine energy there, and forcing a non-overlapping split would misrepresent the underlying physics as well as erase the exact spectral-overlap failure mode this baseline exists to expose. Filter order and type were *not* tuned against the evaluation data — doing so would reproduce, in miniature, the leakage problem Section 4 exists to prevent, since it would fit the “zero-training” baseline to the very mixtures it is scored against.

6.5 Baseline 2: supervised NMF

Baseline 1 answers “is the pipeline sane.” It says nothing about whether an actual published separation method works on this dataset, so `src/baselines.py` also implements a leakage-safe reproduction of a supervised non-negative matrix factorization (NMF) method from the literature identified during related-work review ([7]; Section 11), following the classic supervised-NMF separation recipe of Smaragdis (2007): two fixed dictionaries, one per source, are learned once from isolated recordings via KL-divergence multiplicative-update NMF (Lee & Seung 2001), then frozen

and used to solve per-mixture activations against a held-out mixture. Hyperparameters (dictionary ranks $K_i=10$ heart / $K_r=20$ lung, 100 MU iterations to fit each dictionary, 60 MU iterations to solve activations) were given during this project’s method handoff as the paper’s own values; they have not yet been independently cross-checked against the primary source (Section 13). Separation is a soft, Wiener-style mask applied to the mixture’s complex STFT ($N_{\text{fft}}=512$, hop 128), built from the two sources’ NMF-reconstructed magnitude spectra.

Leakage trap. A naïve reproduction fits both dictionaries on “all of `HS.csv/LS.csv`.” Section 4 shows this lets a third of those recordings — some of them a held-out mixture’s own ground truth — into the dictionary the mixture is later scored against. `make_supervised_nmf_baseline()` instead receives only each fold’s leakage-safe `hs_allowed/ls_allowed` pool from `eval_harness.cross_validate` (Section 6.2) — the same restriction Baseline 1 is exposed to, though bandpass has no dictionary to leak into. A leakage-safe score for this method is therefore *expected* to come in lower than a naïve reproduction’s; that lower number, not a higher one, is the methodologically correct result (Section 7).

6.6 Baseline 3: standard NMF (dictionary ablation)

Baseline 2 conflates two effects: NMF factorization itself, and the specific pretrained/frozen dictionary it factorizes against. Baseline 3 isolates the second effect by holding every other stage of Baseline 2’s pipeline fixed (same total rank $K_i+K_r=30$, same STFT parameters, same Butterworth denoise pre-filter) while factorizing W and H directly out of each held-out mixture’s own spectrogram, with no dictionary-learning phase from isolated recordings at all — so, like Baseline 1, it never touches `hs_allowed/ls_allowed` and has nothing to leak. The resulting components are unlabeled by construction; they are assigned to heart or lung post-hoc by spectral centroid against Baseline 1’s own PSD survey (heart energy concentrated below ~ 200 Hz), a fixed physical prior rather than a source label, so the method stays genuinely blind. The comparison this buys is direct: any gap between Baseline 2 and Baseline 3 is attributable to the learned, frozen dictionary specifically, not to NMF factorization or the shared preprocessing.

6.7 Baseline 4: multi-stage Singular Spectrum Analysis (MSSA)

Baseline 4 reproduces the two-stage MSSA method of Han & Quan [9], the same paper whose own Table I is reported alongside this project’s results in Section 7 for direct comparison. All four hyperparameters are taken as stated in the paper rather than tuned: window length $L=50$, cardiac/respiratory split at 250 Hz (a physiological frequency in absolute Hz, valid at this dataset’s 4000 Hz sample rate since Nyquist, 2000 Hz, clears it with margin), an eigenvalue contribution threshold of $100/L = 2\%$, and a correlation threshold of 50%. Stage 1 SSA-decomposes the raw mixture into 50 reconstructed components (RC), routing each by its dominant Welch-PSD frequency into either the cardiac output directly (≤ 250 Hz) or a residual pool feeding Stage 2; Stage 2 re-decomposes that residual and selects respiratory components by eigenvalue contribution, then Pearson correlation against the already-selected set. Like Baseline 1, MSSA is zero-training — `hs_allowed/ls_allowed` go unused and no leakage trap applies, since there is no fitting step at all.

6.8 Baseline 5: EVMD (Enhanced Variational Mode Decomposition)

Baseline 5 reproduces the EVMD-based lung-isolation stage of Puneet et al. [8], the only related-work paper in Table 27 whose separation step reports no separation-quality metric at all despite running directly on HLS-CMDS mixtures — Table 11 below is, to this project’s knowledge, the first SDR/SIR/SAR computed for that method on this dataset. Variational Mode Decomposition

itself (Dragomiretzkiy & Zosso, 2014) is implemented from scratch (no maintained VMD package is available), and the paper’s own K -selection sweep ($K=2..10$ at $\alpha=2000$, stopping once an energy-loss criterion and a per-mode check both pass) is followed as stated. The four-criterion per-mode check the paper describes (frequency-domain signature, NPE ratio, and a Kurtosis-Index fallback) is under-specified in the primary source; this project’s best-effort reading of how the criteria combine is documented inline in `code_description.md` rather than presented as a literal reproduction. Like Baselines 1 and 4, EVMD is zero-training. The full K -sweep costs ~ 15 s/mixture, dominated by VMD’s ADMM iterations; Section 7’s synthetic-substrate column for this baseline therefore runs on a stratified 75-row subsample rather than the full 1500-row synthetic set, a disclosed compute-driven scope reduction rather than a silent one.

6.9 Baseline 6: Conv-TasNet-lite

Baselines 1–5 are either zero-training or a fixed-dictionary fit. Baseline 6 (`convtasnet.py`) is this project’s first neural, end-to-end learned separation baseline, and the first place a training loop with epochs and an optimizer appears in this codebase at all. It uses Conv-TasNet’s own encoder/TCN-separator/decoder design [4], sized down (“lite”, ~ 325 K parameters vs. Conv-TasNet’s own ~ 5 M) for this dataset’s much smaller training pool, with a sigmoid mask per source (no unit-sum constraint, following Conv-TasNet’s own ablation finding that sigmoid performs at least as well as a softmax constraint) and a fixed source order rather than permutation-invariant training, since heart and lung are distinguishable classes here rather than interchangeable speakers.

Sample rate. This dataset is natively 4000 Hz; public Conv-TasNet/Sepformer checkpoints are trained at 8–16 kHz and do not apply without resampling, and resampling up invents no real information. The model is trained from scratch at the native 4000 Hz instead — not a resolution limitation for this task, since heart (20–200 Hz) and lung (100–1000 Hz) energy (Section 6.4’s own PSD survey) sit comfortably under 4000 Hz’s 2000 Hz Nyquist.

Training loop. A fresh network is trained per fold from that fold’s leakage-safe `hs_allowed/ls_allowed` pool — the same contract every other baseline’s `fit_and_separate_fn` receives (Section 6.2). Since Conv-TasNet needs paired (mixed, heart, lung) training examples rather than a spectral dictionary, training mixtures are synthesized on the fly using Section 6.3’s own mixing recipe, so train-time mixtures are drawn from the same distribution the model is evaluated against. A file-level 80/20 split inside the allowed pool (never touching the outer fold’s held-out data) gives an internal validation set for early stopping and learning-rate scheduling (AdamW, LR halved after 4 epochs without validation-SI-SDR improvement, best-checkpoint restore).

Honest scope. This is the first neural baseline, establishing the training-loop infrastructure — not a tuned, converged model. The dataset’s ~ 50 recordings/class (fewer per fold once leakage exclusions apply) is small for a from-scratch neural separator even with on-the-fly mixing augmentation. Rather than assume the shipped configuration (`MAX_EPOCHS=25`, `STEPS_PER_EPOCH=40`, `BATCH_SIZE=8`) trained to convergence, `baseline6_report.py` reports each fold’s actual training diagnostics and measured wall-clock directly (Table 12).

7 Results

Per Section 5, every baseline is reported on two row sets: the full 145-row `Mix.csv` (Table 9, for continuity with the fold sizes in Table 5) and the 36-row additive-valid subset alone (Table 10). All numbers are 5-fold cross-validated (`eval_harness.cross_validate`, seed 0), reported as across-fold mean $\pm$ std (fold-level dispersion, Section 6.2); Table 10 additionally gives the pooled row-level

mean $\pm$ 95% CI for the valid subset alongside the fold mean, since the two conventions differ and both appear in this report.

7.1 Full 145-row set

Table 9 reports both baselines alongside a no-separation reference (the raw mixed signal returned unmodified as both the heart and lung estimate, i.e. what a classifier would see with no separation step at all).

Table 9: 5-fold cross-validated separation quality on the full 145-row set: both baselines vs. no separation.

Method	Source	SDR (dB)	SIR (dB)	SAR (dB)
No separation	Heart	-13.68 ± 1.26	1.71 ± 0.93	6.46 ± 3.71
No separation	Lung	-15.89 ± 0.84	-1.55 ± 0.91	6.46 ± 3.71
Baseline 1 (bandpass)	Heart	-12.65 ± 1.40	4.07 ± 1.16	-8.10 ± 1.44
Baseline 1 (bandpass)	Lung	-13.83 ± 1.12	1.73 ± 0.29	-9.27 ± 1.42
Baseline 2 (supervised NMF)	Heart	-12.82 ± 1.31	4.22 ± 1.63	-9.58 ± 0.90
Baseline 2 (supervised NMF)	Lung	-15.09 ± 0.98	0.32 ± 0.58	-9.23 ± 1.42

(SAR is identical for heart and lung under “no separation” because with no processing applied, there is nothing to distinguish one source’s artifacts from the other’s.)

Bandpass filtering improves SIR for both sources (+2.4 dB heart, +3.3 dB lung) — it does separate on frequency grounds — but SAR collapses by roughly 14–16 dB relative to doing nothing, consistent with filter ringing and destroyed harmonics dominating the residual error. SDR, which folds interference and artifacts together, only nets a modest ~ 1 –2 dB gain over the no-separation reference as a result. This matches the theoretical interference/artifact trade-off discussed in Section 11.2 and is exactly the outcome a bandpass sanity-floor baseline is expected to produce: real but limited separation, purchased at a real artifact cost. Baseline 2 (leakage-safe supervised NMF) lands essentially tied with Baseline 1 on heart SDR (within one fold-std) and ~ 1.3 dB worse on lung SDR — a learned, source-specific dictionary barely matching, and on lung losing to, a filter with zero learned parameters.

But 109 of these 145 rows are non-additive under the named-reference test (Section 5): for those rows there is no real heart+lung structure in `mixed` for any method to recover, so a method’s score on them measures how it responds to essentially arbitrary audio, not separation quality. Table 9 is retained for continuity with the split reported in Table 5, but the 36-row subset below is the number this report treats as meaningful.

7.2 36-row additive-valid subset

Every number in Table 10 is computed from one file, `results/canonical_rows.csv` (`canonical_rows.py`): the BSS Eval scores of the cached separated waveforms of the S6-02 run (`results/sdr_sweep_cache/`, 16-bit PCM, 2026-08-28), which are also the exact audio Condition B and every knee-point curve consume. Two summaries of the same 36 rows are given side by side because both are in use in this report and they differ: the across-fold mean of per-fold means $\pm$ its std over the five folds (8/7/7/7/7 rows), and the pooled mean over the 36 rows $\pm$ a 95% *t*-interval over rows. An earlier revision printed the fold-mean SDR here and the pooled SDR in Condition B’s table (5.61 vs.

5.46 dB for bandpass) without saying so; it also carried a Baseline 2 entry (3.78/−0.44 dB) from a run that predated the primary-source corrections of Section 6.5, and a Baseline 6 entry (5.15 dB) from a separate training run of the same seed, which on the GPU is not bit-reproducible (5.02 dB in the run cached here). All three are superseded by this table.

Table 10: Separation quality on the 36-row additive-valid subset, leakage-safe 5-fold, from `canonical_rows.csv`. SDR is given under both conventions; SIR and SAR are pooled means. Baseline 0 scores the raw mixture as both estimates.

Method	Source	SDR (dB)		SIR (dB)	SAR (dB)
		fold mean $\pm$ std	pooled $\pm$ 95% CI		
Baseline 0 (raw mixture)	Heart	2.65 ± 4.34	2.48 ± 2.79	2.48	74.6
Baseline 0 (raw mixture)	Lung	-2.14 ± 4.19	-1.98 ± 2.66	−1.98	74.6
Baseline 1 (bandpass)	Heart	5.61 ± 3.68	5.46 ± 2.66	6.30	17.47
Baseline 1 (bandpass)	Lung	3.94 ± 3.57	4.05 ± 2.37	5.73	12.42
Baseline 2 (supervised NMF)	Heart	2.69 ± 2.96	2.58 ± 1.87	7.27	7.58
Baseline 2 (supervised NMF)	Lung	0.62 ± 3.14	0.71 ± 2.34	2.36	9.93
Baseline 3 (standard NMF)	Heart	3.44 ± 2.02	3.35 ± 1.88	6.97	9.37
Baseline 3 (standard NMF)	Lung	2.22 ± 3.91	2.31 ± 2.81	5.02	9.66
Baseline 4 (MSSA)	Heart	5.17 ± 4.05	5.02 ± 2.91	5.73	18.81
Baseline 4 (MSSA)	Lung	5.32 ± 3.20	5.39 ± 1.87	8.58	12.18
Baseline 5 (EVMD)	Heart	3.73 ± 3.43	3.61 ± 2.35	4.99	13.99
Baseline 5 (EVMD)	Lung	0.36 ± 3.39	0.50 ± 2.44	1.40	13.56
Baseline 6 (Conv-TasNet-lite)	Heart	5.19 ± 4.14	5.02 ± 2.77	6.28	16.45
Baseline 6 (Conv-TasNet-lite)	Lung	2.05 ± 3.20	2.13 ± 2.21	4.77	9.07

Two things stand out on the valid subset. **First, absolute separation quality is far higher than Table 9 suggests:** SDR jumps by 15–18 dB for both methods and both sources once rows with no genuine mixture structure are excluded, confirming that the full-145-row numbers were dominated by rows where nothing coherent could be separated, not by a uniformly hard separation problem. **Second, and more consequentially, the ranking between the two baselines reverses relative to Table 9:** on the valid subset, zero-training bandpass *beats* leakage-safe supervised NMF by ~ 2.9 dB SDR on heart and ~ 3.3 dB SDR on lung (fold means), a materially larger and more one-sided gap than the near-tie observed on the full (mostly non-additive) row set. A learned, source-specific separation method failing to beat a fixed filter with zero learned parameters, on exactly the rows where genuine separation is possible, is not the result a naïve reading of the reproduced paper would predict, and is discussed further in Section 12.

7.3 Six methods, three substrates

Sections 6.6–6.9 add four more baselines, and Section 6.3 adds the 1500-row synthetic set as this project’s primary evaluation substrate going forward, with the 36-row native-additive subset retained alongside as a distinct column rather than folded in — agreement or disagreement between the two substrates is itself audit evidence, not noise to average away. All numbers below are mean $\pm$ 95% CI; Baseline 5’s synthetic column runs on a stratified 75-row subsample (Section 6.8), not the full 1500.

Table 11: Synthetic substrate (primary), 5-fold, $n=1500$ except Baseline 5 ($n=75$).

Method	Source	SDR (dB)	SIR (dB)	SAR (dB)
Baseline 1 (bandpass)	Heart	2.10 ± 0.43	6.17 ± 0.47	11.01 ± 0.45
Baseline 1 (bandpass)	Lung	-1.24 ± 0.44	4.25 ± 0.40	5.92 ± 0.48
Baseline 2 (supervised NMF)	Heart	-0.70 ± 0.40	6.54 ± 0.44	3.47 ± 0.34
Baseline 2 (supervised NMF)	Lung	-3.73 ± 0.40	1.39 ± 0.39	3.59 ± 0.38
Baseline 3 (standard NMF)	Heart	-0.00 ± 0.43	6.93 ± 0.50	4.84 ± 0.37
Baseline 3 (standard NMF)	Lung	-3.53 ± 0.46	4.06 ± 0.39	1.39 ± 0.46
Baseline 4 (MSSA)	Heart	1.59 ± 0.47	5.58 ± 0.47	11.48 ± 0.52
Baseline 4 (MSSA)	Lung	-1.60 ± 0.50	6.24 ± 0.42	3.74 ± 0.55
Baseline 5 (EVMD)	Heart	2.42 ± 1.70	6.20 ± 2.02	10.15 ± 1.48
Baseline 5 (EVMD)	Lung	-2.99 ± 2.25	2.81 ± 1.90	5.63 ± 2.53
Baseline 6 (Conv-TasNet-lite)	Heart	3.16 ± 0.40	6.47 ± 0.51	12.14 ± 0.31
Baseline 6 (Conv-TasNet-lite)	Lung	0.37 ± 0.35	4.29 ± 0.41	6.96 ± 0.27

The native-additive column of this comparison is Table 10 above (its pooled convention matches the synthetic table’s).

Substrate agreement. Every baseline scores lower on the synthetic substrate than on the 36-row native set (heart gaps -1.2 to -3.4 dB, lung gaps -1.8 to -7.0 dB, computed as synthetic pooled mean minus native pooled mean), consistently in the same direction across all six methods. The synthetic set’s much larger n makes its own CI tight regardless of this gap, so a tight synthetic-column CI is not itself evidence the two substrates agree — the gap column is the evidence, and it says the synthetic set is harder, not that either substrate is unreliable. The lung-side gap is consistently larger than the heart-side gap for every method, which is consistent with Baseline 1’s own PSD survey (Section 6.4): lung energy is more spread and closer to the noise floor, so it is more sensitive to the synthetic set’s noise sweep than heart energy is.

Baseline 6 still does not clearly win. Conv-TasNet-lite scores highest of the six methods on synthetic heart and lung SDR, and highest on native-additive heart SDR, but on native-additive lung SDR it trails Baseline 4 (MSSA, 5.39 vs. 2.14 dB) by more than either method’s CI half-width. A learned, end-to-end model that only sometimes beats a zero-training method is the expected outcome of an intentionally undertrained first neural baseline (Section 6.9), not evidence the architecture is unsuited to the task; Table 12 below is what determines whether “undertrained” is actually the right explanation.

Table 12: Baseline 6 training diagnostics, synthetic-set folds (`max_epochs=25`, `steps_per_epoch=40`, `batch_size=8`).

Fold	Epochs trained	Best val. SI-SDR (dB)	Device	Fit time (s)
0	25	0.78	cuda	27.1
1	25	1.78	cuda	26.2
2	25	2.65	cuda	26.2
3	25	3.35	cuda	26.3
4	25	1.68	cuda	26.2

Every fold trains for the full 25-epoch budget without the LR-halving criterion (4 epochs without validation-SI-SDR improvement) ever firing — best-validation-SI-SDR is still improving, or at worst plateauing briefly, when training stops, which reads as compute-budget-limited rather than converged. Combined with only ~ 32 training recordings per class per fold (Table 5-scale pool sizes, further split 80/20 for internal validation), this supports Section 6.9’s own “honest scope” framing: Baseline 6 establishes that the training-loop infrastructure works end-to-end (Table 11 shows finite, non-degenerate output), not that this architecture has been given a fair chance to demonstrate its ceiling on this dataset.

8 Downstream Classification: Does Separation Hurt Accuracy?

Every section so far measures *separation quality* (SDR/SIR/SAR). None of it yet answers the question the whole project exists to answer (Section 11.1): does separating heart sound from a mixture, before classifying it, actually cost accuracy, and if so, how much, at what separation quality? Answering that needs a classifier, a way to compare its accuracy on clean vs. separated audio under a controlled split, and — because separation quality is continuous, not a single number — a way to sweep separation quality and trace out a curve rather than compare two fixed points.

8.1 Why one classifier architecture, deliberately, at first

The classifier in this section is described in the project charter as a *measuring instrument*, not a research contribution in its own right: its only job is to reveal whether separation quality and accuracy are related, not to be the best possible heart-sound classifier. That framing is the reason exactly one architecture was built first (Section 8.9 revisits this once the classifier’s own result turns out to matter more than originally planned) — a second architecture only becomes necessary once a reader might reasonably ask “is this result about separation, or about the one classifier you happened to pick,” which is a question about the *measuring instrument*, not about classification research for its own sake.

8.2 The class-grouping decision

HLS-CMDS labels each isolated heart recording with one of ten specific types (Table 2). Classifying all ten directly is not possible under this project’s leak-group split: S4 has only 2 recordings total, and under a 5-fold split built from leak groups (Section 4) there is no guarantee those 2 recordings land in different folds — if they land in the same fold, three of the five folds train with *zero* S4 examples at all, and AV Block and Tachycardia ($n=3$ each) are barely better off. This is exactly the small-class risk Section 3.2.1 flagged before this section was built.

Two fixes were considered: collapse to a binary Normal/Abnormal classification, or group the ten types into a small number of clinically meaningful categories. The categories chosen — Normal, Murmur, Extra Sound, Rhythm Disorder — are not an arbitrary compromise: they are the *exact* four-class scheme Yaqub et al. (Section 11.1) already use for this same dataset, confirmed by reading their primary source directly (their Tables 6–9), and reproduce the same class list their own 89%→41% collapse is measured on. Mapping HLS-CMDS’s ten types onto those four is unambiguous from standard cardiology terminology: Normal stays Normal ($n=9$); the four murmur sub-types (Mid/Late Systolic, Late Diastolic, Early Systolic) become Murmur ($n=24$); S3 and S4 — both extra heart sounds occurring outside the normal S1/S2 pair — become Extra Sound ($n=7$); and Atrial Fibrillation, Tachycardia, and AV Block — all disturbances of the heart’s electrical rhythm or conduction, as opposed to its valves — become Rhythm Disorder ($n=10$). This solves

the small-class problem (the smallest group, Extra Sound at $n=7$, is comfortably large enough to appear in every fold) and, unlike collapsing to binary Normal/Abnormal, keeps this project’s accuracy numbers directly comparable to the 86–89% figures Yaqub et al. report on the very scheme this project’s motivating result is anchored to.

8.3 Architecture 1: MFCC summary statistics + an RBF-kernel SVM

Given the class-grouping decision above and Section 8.1’s “measuring instrument, not a contribution” framing, the classifier itself was chosen to be small, well-understood, and unlikely to overfit a dataset this size (50 recordings total, as few as ~ 35 –40 in a fold’s training split): 13 Mel-Frequency Cepstral Coefficients (MFCCs, a standard, perceptually-motivated compression of a spectrogram into a small number of coefficients per time frame, widely used in speech and audio classification before deep learning), pooled to a single 26-dimensional feature vector per recording by taking the mean and standard deviation of each coefficient over time, fed into a Support Vector Machine (SVM) with a Radial Basis Function (RBF) kernel and class-balanced loss weighting (so the majority Murmur class does not simply dominate the optimization by volume alone). An SVM with a fixed, moderate number of support vectors is far less prone to memorizing a training set this small than a gradient-trained deep model would be, which is precisely the trade-off Section 8.9 tests directly with a second architecture rather than assuming.

Evaluation follows the same leak-group folds every separation baseline uses (`assign_hs_folds()`, extending `split.py`’s existing fold assignment from mixture rows to the isolated `HS.csv` recordings themselves), stratified by class group for any recording never reused inside a mixture, so no fold is starved of a minority class. Table 13 reports the resulting 5-fold cross-validated result — the first accuracy number in this report; every prior result is a dB-scale SDR/SIR/SAR figure.

Table 13: Condition A (Architecture 1, MFCC+SVM): isolated-heart-sound classification on the 50 `HS.csv` recordings, 5-fold cross-validated. Not the same evaluation set as Condition B’s “isolated” row (Table 14), which scores the same per-fold classifiers on the 36 native-additive mixtures’ own heart references; the two accuracies (60.0% vs. 52.8%) differ by design.

Metric	Mean $\pm$ 95% CI ($n=5$ folds)
Accuracy	60.0% $\pm$ 10.7%
Macro-F1	0.43 $\pm$ 0.12

With only 5 folds, this confidence interval should be read as indicative, not decisive (the same caveat Section 6.2 already applies to every separation baseline’s across-fold dispersion) — it says roughly where the accuracy sits, not that a repeat run would land within ± 0.1 percentage points of it.

8.4 Failure-mode analysis: the confusion matrix

An aggregate accuracy number hides *which* classes are actually being confused for which. Section 2.8’s confusion matrix makes this concrete: pooling every fold’s held-out predictions (each recording predicted exactly once, so pooling never double-counts), the per-class recall is Normal 44%, Murmur 92%, Extra Sound 29%, Rhythm Disorder 20%. The dominant confusion is directional, not symmetric: the minority classes’ errors pull toward Murmur — Extra Sound is predicted Murmur *more* often than it is predicted correctly (43% vs. 29% recall), and Rhythm Disorder is predicted Murmur or Extra Sound (30% each) more often than itself (20%) — while Murmur is

almost never mistaken for anything else (0% confused as Normal, 4% each as Extra Sound and Rhythm Disorder). Normal is the one exception, whose errors run toward Rhythm Disorder (33%) instead. This is consistent with Murmur’s training-set dominance ($n=24$ of 50) pulling the SVM’s decision boundary toward it despite class-balanced loss weighting — reweighting the training loss does not guarantee balanced *predictions* once the feature space itself gives the four classes limited separability to begin with.

8.5 A controlled way to sweep separation quality: the degradation scheme

Comparing accuracy at exactly two points — perfectly isolated audio, and one separation method’s real output — is what Yaqub et al. do (Section 11.1), and the charter’s own C2 research question asks for more: not just *whether* separation costs accuracy, but the *knee point* — the separation quality (SDR) below which separation starts actively hurting accuracy relative to not separating at all. A knee point cannot be identified from two endpoints; it needs a continuous curve across a range of separation quality.

`synthetic_mix.py`’s noise sweep (Section 6.3) already varies difficulty *before* separation runs, by controlling how noisy the input mixture is. A second, complementary knob varies difficulty *after* separation: take a real separation method’s actual output s and its ground truth g , and blend them,

$$\text{degraded}(\alpha) = (1 - \alpha)g + \alpha s = g + \alpha(s - g), \quad \alpha \in [0, 1], \quad (9)$$

so $\alpha=0$ reproduces the clean ground truth exactly and $\alpha=1$ reproduces that method’s real separated output exactly. This has the same “signal plus a scaled error term” shape as the noise sweep in Equation 1’s synthetic-mixing recipe, except the injected term $s - g$ is a *real separation method’s own characteristic error* (its own specific mix of interference and artifact, Section 2.2) rather than generic Gaussian noise — the two knobs are kept side by side deliberately, because they answer different questions: the noise sweep asks “how does accuracy degrade as the separation *problem* gets harder,” while the interpolation sweep asks “at a given separation quality, however it was reached, how does accuracy behave,” holding one specific method’s error *shape* fixed and varying only its magnitude.

A real, measured caveat about this scheme, found rather than assumed. Validating Equation 9 on real audio revealed that a *uniform* grid of α values does *not* give an even spread of measured SDR: on the row checked, SDR fell from ~ 262 dB at $\alpha=0$ (near-bit-identical to the clean signal) to ~ 10.6 dB by $\alpha=0.1$ alone, then flattened out to ~ 4.8 dB by $\alpha=1$ — because decibels are logarithmic and most sensitive to a small *absolute* error right where the signal is nearly clean. A uniform α grid therefore wastes most of its points on the flat tail. The first implementation therefore solved for α by bisection, on the assumption — checked on one row, not proved — that SDR is monotone in α . That assumption was later audited on every curve and found to fail for seven of them, and the bisection has been replaced; both are described here because the sweep’s validity rests on it.

SDR(α) has a closed form, so the audit is exhaustive. Equation 9 is linear in α , and BSS Eval’s decomposition of an estimate into target and error components is a fixed linear projection onto the span of the delayed references. Every component of $d(\alpha)$ is therefore the same convex combination of the components of g and of s , and $\text{SDR}(\alpha) = 10 \log_{10} N(\alpha)/D(\alpha)$ with N and D quadratics in α whose six coefficients are inner products of the two decompositions. Two BSS Eval decompositions per (method, row, source) give the exact curve at any resolution (`sdr_alpha_audit.py`); the closed form was checked against `mir_eval` at five interior α values on 72 curves and agrees to 10^{-5} dB wherever `mir_eval`’s permutation search keeps the heart/lung assignment. On a 1001-point grid, 425 of the 432 curves (six methods, 36 rows, two sources) are

monotone non-increasing to within 10^{-6} dB. The seven that are not are all Conv-TasNet-lite heart curves: the network’s estimate on those rows is close to a polarity-inverted copy of the reference, so the blend passes near cancellation and SDR dips to between -6 and -20 dB at $\alpha \approx 0.04$ – 0.17 before rising to the method’s real output (-7 to $+19$ dB). For those rows targets below the real output have two roots, and the bisection, which only ever probed $\alpha \geq 10^{-3}$ on a log grid and assumed one crossing, clamped them to the real output — a silently wrong point. On the monotone curves the bisection was adequate: median $|\alpha_{\text{bisection}} - \alpha_{\text{exact}}| = 2 \times 10^{-4}$, 90th percentile 3.5×10^{-3} , and the achieved SDR within 0.17 dB of the target at the 90th percentile.

The sweep now solves for α exactly, under a pre-stated rule. Since $\text{SDR}(\alpha) = t$ is the quadratic $N(\alpha) - 10^{t/10} D(\alpha) = 0$, every root in $[0, 1]$ is found in closed form, with no grid resolution to worry about even where the curve falls 230 dB in the first 10^{-3} of α . The rule, fixed before the re-run: take the *smallest* root — the crossing on the branch that starts at the ground truth — and record the number of roots and whether the row’s curve is monotone in the provenance. A target with no root in $[0, 1]$ (above $\text{SDR}(0)$, or below the curve’s minimum, which for a monotone curve is the method’s own output) is recorded as unattainable and the point is placed at $\alpha=1$ with its clamp flagged, as before; it is never searched for. Each sweep point’s SDR is scored with the heart/lung assignment fixed, because inside a cancellation dip the heart estimate carries almost no energy and `mir_eval`’s permutation search would score the lung estimate as “heart” instead; the permuted value is stored alongside and flagged where it differs. Re-generating the provenance from the cached separated audio under this rule changed the α of 99 of 1053 attainable heart targets by more than 0.01 — 22 of them on the seven non-monotone rows, the rest where the bisection’s 0.1 dB tolerance or a permutation flip in its measured SDR had moved it — and un-clamped 55 points; every attainable point now sits at its target to four decimals; every curve, knee and bootstrap figure in this report is from the re-generated sweep, and the point grid remains $\{25, 20, 15, 10, 5, 0, -5\}$ dB.

8.6 Generating the SDR sweep

For every one of the six separation baselines, every one of the 36 native additive rows (Section 5), and both sources, the sweep generator (`sdr_sweep.py`) fits that baseline once per fold (the same leakage-safe contract every earlier baseline uses, Section 6.2), separates the held-out mixture, caches the real separated audio to disk, and then applies Section 8.5’s target-SDR root-finding to hit every value in the target grid — one separation call per row, then seven cheap degraded points reconstructed from it, rather than seven expensive re-separations. This produces up to $6 \times 36 \times 2 \times 7 = 3024$ provenance rows, each recording which baseline, which mixture, which source, what SDR was targeted, what α actually achieved it, and the resulting measured SDR/SIR/SAR.

Because this involves re-running every separation baseline — including the two most expensive, EVMD (Section 6.8, ~ 15 s per mixture) and Conv-TasNet-lite (Section 6.9, which needs a freshly trained network per fold) — generation is scripted to checkpoint incrementally and resume safely if interrupted, rather than needing to run start-to-finish in one sitting. The generation run is complete: all 3024 provenance rows exist (six baselines $\times$ 36 rows $\times$ two sources $\times$ seven targets, five folds each), and the cache-then- reconstruct path was re-checked on 20 sampled rows against the recorded achieved SDR with a maximum discrepancy of 0.25 dB. One property of the sweep matters for reading every curve below: the target grid runs from -5 to 25 dB, but a point cannot be degraded *below* the baseline’s own real output, so targets under that floor are clamped to it (29–37% of rows per baseline). Each baseline’s curve therefore starts at its own real SDR — 2.6 dB for Baseline 2, 5.5 dB for Baseline 1 — and its two lowest target points coincide with its Condition B operating point.

8.7 Condition B: classifying separated audio, and the weight-sharing decision

A subtlety has to be settled before “classify the separated audio” is well-defined: is the classifier used on separated audio *the same trained weights* as Condition A, or a fresh classifier retrained on separated training audio? These are different experiments answering different questions — the former isolates “does separation hurt a classifier that never saw separation artifacts during training,” the latter asks “can a classifier adapt to separation artifacts if it is allowed to.” This project uses the *same trained weights* per fold, matching Yaqub et al.’s own methodology exactly (their Experiment 3 model *is* their Experiment 4 model, with no retraining in between, Section 11.1): reproducing their finding under controlled conditions means matching their causal claim, and same-weights is also the cleaner isolation of the separation method as the only variable between the two conditions.

A second, easy-to-miss consistency requirement follows: the classifier’s fold assignment and the sweep’s fold assignment must agree on what “fold k ” means, or a fold’s classifier could unknowingly be evaluated on separated audio derived from a recording it was partly trained on. Because `sdr_sweep.py` computes its folds over the 36-row native-additive subset specifically (not the full 145-row `Mix.csv` Condition A’s own standalone fold assignment defaults to), the classifier’s fold assignment is explicitly rebuilt from that same 36-row basis for Condition B, rather than assumed to already match.

A confound found in the first run, and removed. The first complete Condition B run produced a result that looked like a finding and was a defect: Baselines 1, 4 and 6 — a fixed filter, an SVD decomposition and a neural network — returned the *same 36 predictions*. The cached separated audio was checked first and was distinct per method (different content hashes, different RMS), and the loading path resolves each baseline to its own directory, so the classifier was genuinely seeing different audio and returning the same answer. The reason was level. `HS.csv`’s recordings span 0.0005–0.013 RMS, while every baseline’s heart estimate inherits the mixture’s peak-normalising gain and sits at 0.03–0.09 RMS — 20–25 $\times$ louder than anything the classifier had trained on. MFCC₀ is log-energy, so separated inputs landed at a standardised distance of 11–15 from the origin against a training cloud whose farthest point is at 7.4; an RBF kernel evaluates to ≈ 0 that far from every support vector, and the SVM’s decision function collapsed to the same vector ([0.95, 3.13, -0.16, 2.10] for bandpass and Conv-TasNet alike) for every far-away input. The prediction was a per-fold constant — Rhythm Disorder for fold 0’s classifier, Murmur for the other four — and did not depend on the audio. Worse, the sweep’s blend $(1 - \alpha) \text{ref} + \alpha \text{est}$ moved loudness back toward the training range as SDR rose, so the first accuracy-vs-SDR curve was partly a loudness-vs-SDR curve.

The fix is to make both classifiers invariant to absolute level: every waveform is scaled to unit RMS before feature extraction (`heart_classifier.level_normalize`, applied identically by both backends at training and at inference). Nothing else changed. After it, isolated references sit inside the training cloud (mean standardised magnitude ≈ 1 per feature) and separated estimates remain outside it — at distance 8–13, now on the spectral-shape coefficients (MFCC₁–MFCC₄) rather than on energy — which is the distortion Condition B exists to measure: a 20–200 Hz bandpass removes most of the spectral envelope a heart-sound classifier trained on full-band recordings relies on. Condition A also changed with the retrained classifiers (Table 13 reports the post-fix values); every Condition B and knee-point number in this report is from the corrected run, and the pre-fix run is kept only in this paragraph.

Table 14 reports the result for all six baselines on the same 36 rows, with the same per-fold weights throughout.

Table 14: Condition B, both architectures, 36 native additive rows, same per-fold weights as Condition A. The isolated row is those 36 rows’ own heart references scored with the same per-fold classifiers — a different, smaller set than Condition A’s 50 recordings (Table 13), which is why it reads 52.8% here and 60.0/62.0% there. Accuracy is pooled over the 36 rows; every interval is a 95% percentile interval from 5000 cluster-bootstrap replicates over the 24 leak groups the rows fall into (`condition_b_stats.py`). Δ is isolated minus separated accuracy, paired by row; Δ_{raw} is separated minus raw-mixture accuracy, the knee question at one point. Mean SDR is each method’s real heart output (Table 10, pooled).

Arch.	Input	Accuracy [95% CI]	SDR	Δ vs. isolated [95% CI]	Δ_{raw} [95% CI]
1	Isolated ground truth	52.8 [35.3, 70.0]	—	—	—
1	Raw mixture (no separation)	38.9 [24.1, 53.1]	2.48	+13.9 [0.0, 29.0]	—
1	Baseline 1 (bandpass)	30.6 [14.3, 45.2]	5.46	+22.2 [2.5, 45.2]	−8.3 [−28.6, 8.1]
1	Baseline 2 (supervised NMF)	41.7 [24.3, 57.9]	2.58	+11.1 [−8.6, 33.3]	+2.8 [−17.1, 20.0]
1	Baseline 3 (standard NMF)	27.8 [14.3, 39.5]	3.35	+25.0 [11.6, 41.9]	−11.1 [−24.1, −2.4]
1	Baseline 4 (MSSA)	36.1 [19.4, 51.2]	5.02	+16.7 [−2.4, 38.9]	−2.8 [−20.6, 11.4]
1	Baseline 5 (EVMD)	27.8 [10.7, 43.2]	3.61	+25.0 [3.2, 48.6]	−11.1 [−32.4, 7.4]
1	Baseline 6 (Conv-TasNet-lite)	36.1 [21.2, 51.4]	5.02	+16.7 [−2.6, 38.2]	−2.8 [−23.5, 16.2]
2	Isolated ground truth	52.8 [34.3, 69.2]	—	—	—
2	Raw mixture (no separation)	33.3 [17.9, 50.0]	2.48	+19.4 [−2.7, 38.9]	—
2	Baseline 1 (bandpass)	16.7 [3.1, 30.6]	5.46	+36.1 [19.4, 52.8]	−16.7 [−34.5, −2.6]
2	Baseline 2 (supervised NMF)	44.4 [29.0, 57.6]	2.58	+8.3 [−10.5, 25.0]	+11.1 [−6.9, 27.8]
2	Baseline 3 (standard NMF)	27.8 [11.8, 44.4]	3.35	+25.0 [7.4, 40.5]	−5.6 [−22.9, 12.1]
2	Baseline 4 (MSSA)	16.7 [3.1, 30.6]	5.02	+36.1 [19.4, 52.8]	−16.7 [−34.5, −2.6]
2	Baseline 5 (EVMD)	16.7 [3.1, 30.6]	3.61	+36.1 [19.4, 52.8]	−16.7 [−34.5, −2.6]
2	Baseline 6 (Conv-TasNet-lite)	36.1 [19.4, 52.9]	5.02	+16.7 [−3.4, 34.4]	+2.8 [−18.4, 23.5]

The direction matches Yaqub et al.’s collapse for every one of the six methods under both classifiers, and the intervals say how much of it is resolvable at $n=36$. Under Architecture 1 the drop is 11–25 points, and the leak-group bootstrap interval of the paired difference excludes zero for three methods (bandpass, standard NMF, EVMD); a group-level sign-flip permutation test gives $p = 0.075, 0.004$ and 0.062 for the three, and 0.18 – 0.43 for the rest. Under Architecture 2 the drop is 8–36 points and excludes zero for four (bandpass, standard NMF, MSSA, EVMD; permutation $p = 0.003$ – 0.031). The earlier fold-level paired t -tests ($n=5$) are retained in `condition_b_report.html` but are not the basis of any claim: five fold means of unequal size are a weaker and less honest unit than 36 paired rows resampled by leak group. The Δ_{raw} column asks the knee question at one point — is the separated audio classified *better than the untouched mixture*? — and the answer is no for every method under both classifiers: every interval includes zero or lies below it, and for standard NMF (Architecture 1) and for bandpass, MSSA and EVMD (Architecture 2) the whole interval is below zero.

Two things in the table run against a naive reading of the SDR column. The *smallest* drop belongs to the method with the *lowest* SDR: supervised NMF, at 2.6 dB, loses 11 (Architecture 1) and 8 points (Architecture 2), where bandpass, at 5.5 dB, loses 22 and 36. And the three methods that strip spectral content — bandpass, MSSA and EVMD — produce nearly the same predictions as one another under the SVM (31–34 of 36 identical) and *exactly* the same 36 predictions under the CNN, which scores all three at 16.7%. That last number is a per-fold constant again: the CNN’s logits on those inputs differ by method but share the same arg-max in every fold (Extra Sound in folds 0 and 3, Normal in the others), so a classifier trained on full-band recordings, faced with audio from which most of the spectral envelope has been removed, falls back to a default

class whatever the method. Level normalisation removed one cause of this collapse (Section 8.7); the spectral one is not a defect but the phenomenon Condition B measures, and it means that at their real output the three stripping methods are indistinguishable to either classifier — the accuracy differences between them in the table are differences of zero to three rows. The classifier is evidently responding to how much of the spectral envelope survives separation, which SDR, a waveform-domain energy ratio, does not measure directly. This table reproduces the direction of Yaqub’s result at roughly half its size; the trajectories below are what the C2 question actually needs.

8.8 The accuracy-vs-SDR curve and the knee point

Once every target-SDR point in the sweep (Section 8.6) has been classified with Condition B’s weight-sharing setup, the result is a curve per baseline: measured SDR on the horizontal axis, classification accuracy on the vertical axis. This is, as far as this project has found, the first time HLS-CMDS classification accuracy and separation-quality SDR have been placed on the same axis at all — Yaqub et al. report accuracy with no SDR attached to either endpoint, and every separation-quality table in this report (Section 7) reports SDR with no downstream accuracy attached.

Definition of the knee point. Rather than an arbitrary curvature-based “elbow” heuristic, the knee point is defined directly from the charter’s own wording: the SDR at which a baseline’s accuracy curve crosses the accuracy of *not separating at all* (classifying the raw, unseparated mixture directly with the same classifier weights), walking from high SDR down to low SDR. Above the knee, separating helps (or at least does not hurt); below it, a classifier would have done as well or better without separating at all. The crossing point is found by linear interpolation between the two adjacent *measured* points that bracket it — not a smoothed curve fit through all seven points, since smoothing through only seven points (each an average over a handful of rows) would manufacture a precision the data does not support. Four outcomes are possible and are reported distinctly rather than collapsed into one number: a clean crossing; the curve staying above the no-separation reference across the whole measured range (no knee found, separation always helps here); staying below across the whole range (separation never helps); or a “noisy” crossing in the wrong direction, flagged as such rather than silently treated as a clean knee.

The curve. Figure 1 is the complete six-baseline curve for Architecture 1, all 1512 heart-source sweep points classified with Condition B’s per-fold weights; Table 15 gives the knee points the detector found on it.

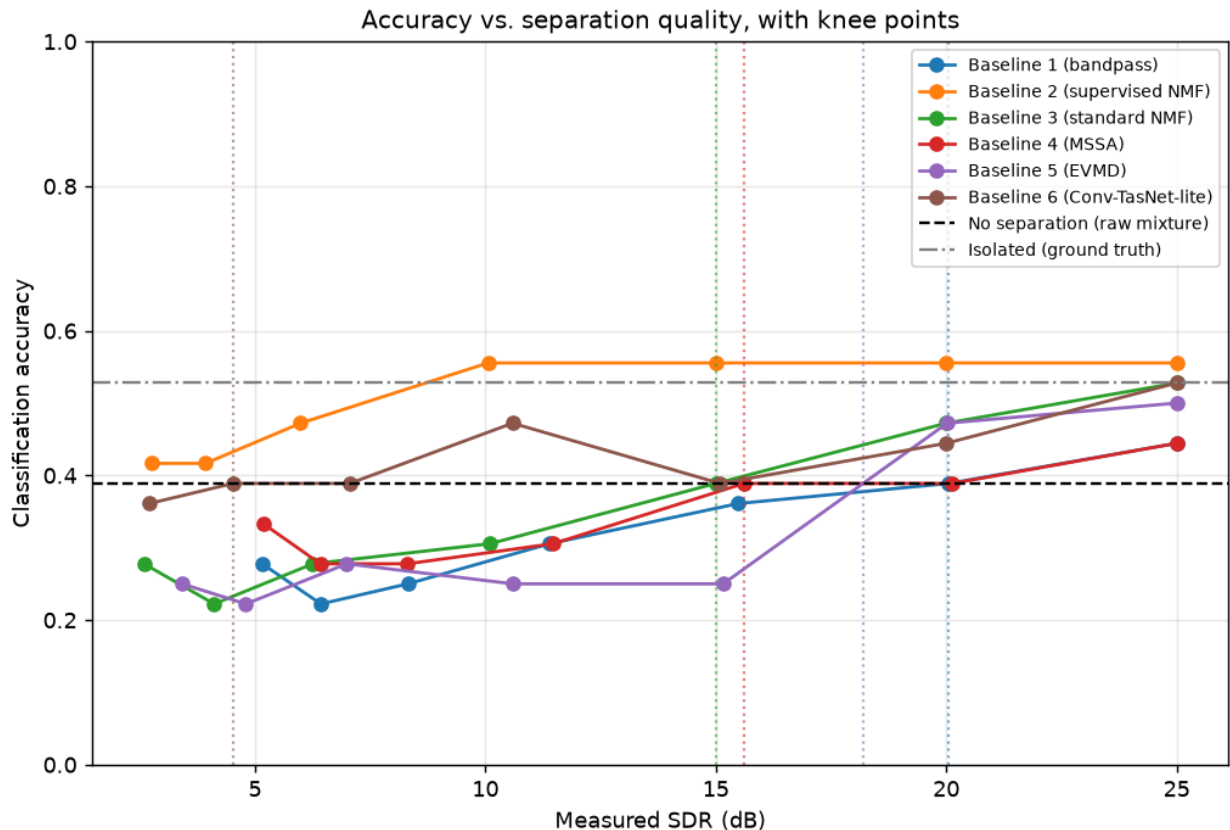


Figure 1: Classification accuracy against measured SDR, Architecture 1 (MFCC+SVM), 36 native additive rows per point. The dashed line is the same classifier on the raw mixture (38.9%); the dash-dot line is isolated ground truth (52.8%). Dotted verticals mark each baseline’s knee point. Each curve’s leftmost points sit at that baseline’s own real output SDR, because the sweep cannot degrade below it.

Table 15: Raw-mixture crossings, Architecture 1: the SDR at which a baseline’s trajectory, walking from high to low SDR, crosses the no-separation reference (38.9%), by linear interpolation between the two bracketing sampled points; “always above” means no sampled point falls below the reference. Bootstrap columns are from Table 18 (row resampling); “n.i.” = not identifiable within the 25 dB grid. Real output SDR is the method’s own output (Table 10, pooled).

Baseline	Crossing (dB)	$P(\text{crossed})$	95% interval	Real output SDR (dB)
Baseline 1 (bandpass)	20.1	0.71	[7.3, n.i.]	5.46
Baseline 2 (supervised NMF)	always above	0.43	—	2.58
Baseline 3 (standard NMF)	15.0	0.90	[5.0, n.i.]	3.35
Baseline 4 (MSSA)	15.6	0.72	[7.3, n.i.]	5.02
Baseline 5 (EVMD)	18.2	0.85	[6.1, n.i.]	3.61
Baseline 6 (Conv-TasNet-lite)	4.5	0.63	[3.1, n.i.]	5.02

What the trajectories show. The six split into two groups that the SDR axis alone would not predict. Bandpass, standard NMF, MSSA and EVMD (Baselines 1, 3, 4, 5) all start at 25–

33% at their own real output — below the 38.9% no-separation reference — and climb to 44–53% at 25 dB, crossing the reference between 15 and 20 dB. Supervised NMF and Conv-TasNet-lite (Baselines 2 and 6) start *at or above* the reference despite having the lowest and a middling SDR respectively: Baseline 2 never falls below it at any sampled point and sits at the isolated ceiling from 10 dB upward, and Baseline 6’s only excursion below it is its two lowest points (36.1% against 38.9%, one row), which the detector reports as a crossing at 4.5 dB. Both are methods whose output is the mixture’s own STFT under a magnitude mask (or a learned resynthesis of it), so the spectral envelope the classifier was trained on survives. Bandpass, MSSA and EVMD discard spectral content by construction; standard NMF is a mask method too, but its mask is fitted on the mixture alone with no source dictionary, and under this classifier it behaves like the three that strip. For those four the figure says what it said before the level fix, now without the confound: at the operating point each actually achieves, its output classifies 6–14 points worse than the untouched mixture, and along this interpolation protocol it would need roughly 15 to 20 dB of heart-source SDR — 10 to 15 dB more than it delivers — before separating beat not separating. For the two mask-based methods the same classifier is indifferent to separation at every SDR: their output never hurts and, at the low SDR they reach, never helps either.

What they do not show. The vertical resolution is one row in 36 (2.8 points), and a binomial 95% interval on a 36-row accuracy near 40% is about ± 16 points, wider than the gap between adjacent trajectories at most SDRs. The bootstrap columns of Table 15 make this concrete: for every baseline the upper end of the crossing’s interval is the top of the measured grid, and for bandpass, MSSA and Conv-TasNet a quarter to a third of resamples have no crossing at all. Individual crossing values within the late-crossing group are therefore not distinguishable from one another, and Baseline 6’s 4.5 dB is a one-row excursion that the CNN below does not reproduce. What survives the noise is the grouping of Baselines 2 and 6 against Baselines 1, 4 and 5 — consistent across all seven target levels and, as Section 8.9 shows, across both classifiers — and the fact that no spectral-stripping method’s operating point is anywhere near its crossing region. The crossing *region* for that group, roughly 15–20 dB under this classifier, is the quotable figure; the individual values are not.

8.9 Robustness check: does a second, different classifier see the same knee?

Section 8.1 explained why one classifier architecture was acceptable while separation quality (not accuracy) was this project’s headline result. That reasoning no longer applies once the knee point itself *is* the headline result: the first question a skeptical reader will ask is whether a knee point is a property of separation quality, or an artifact of the one architecture that happened to measure it. The only way to answer that is to repeat the entire curve with a second, architecturally distinct classifier and see whether the two knee points agree.

“Architecturally distinct” matters here, not just “a second model”: two different SVMs, or an SVM with different hyperparameters, would not isolate the right variable, since both would still share the same feature representation and the same kind of decision boundary. The second architecture built for this check instead uses a log-Mel spectrogram (a 2-D time-frequency image, the same kind of representation Baseline 6’s encoder learns implicitly, Section 2.6) fed into a small convolutional neural network (two convolutional blocks, global average pooling, a single linear output layer) — a gradient-trained model operating on a fundamentally different input shape than Architecture 1’s pooled MFCC statistics, chosen specifically to maximize the chance that a disagreement, if found, reflects something real about the two architectures’ different decision boundaries rather than two flavors of the same idea.

Table 16: Architecture 2 (log-Mel + CNN) vs. Architecture 1 (MFCC+SVM), Condition A, 5-fold cross-validated.

Metric	Architecture 1 (MFCC+SVM)	Architecture 2 (log-Mel+CNN)
Accuracy	60.0% $\pm$ 10.7%	62.0% $\pm$ 3.9%
Macro-F1	0.43 $\pm$ 0.12	0.54 $\pm$ 0.11

The first configuration was underfitting, and the fix was optimisation, not architecture. Architecture 2’s first configuration — raw log-Mel input, 40 full-batch Adam steps — scored 30.0%, far below Architecture 1, with a confusion matrix collapsed toward *Normal*. An earlier revision read that as the small-data overfitting risk of Section 8.3 coming true. It was the opposite: the model’s *training* accuracy was 23–40% per fold, so it had not fitted the training data either. Two causes, both mundane. The log-Mel features sit around -12.4 ± 2.3 (a 10^{-6} floor on quiet recordings), an offset the first convolution had to absorb before learning anything; and 40 full-batch steps is 40 gradient updates in total. Standardising each Mel bin with the training fold’s own mean and standard deviation (stored on the classifier, applied identically at test time, so nothing from a held-out fold enters), training in mini-batches of eight for 300 epochs with weight decay 10^{-3} , and changing nothing about the network, brings training accuracy to 85–95%. Three configurations were run in total and all three are recorded in the module’s docstring; the adopted one was chosen as the first whose training accuracy cleared 85% on every fold — the point at which underfitting was demonstrably gone — and the reader should know it also had the highest test accuracy of the three at the time, and weigh that accordingly. That tuning was done before the level normalisation of Section 8.7 was added to both backends; with it, and no further change, Architecture 2 scores 62.0% $\pm$ 3.9% (fold accuracies 60–70%, the tightest spread of any classifier in this report) with macro-F1 0.54. Its confusion matrix no longer collapses: Normal recall 67%, Murmur 71%, Extra Sound 71%, Rhythm Disorder 30% — a more balanced profile than Architecture 1’s, which buys its 92% Murmur recall with 20–29% on the minority classes.

The second curve. Figure 2 repeats the entire sweep classification with the tuned Architecture 2, on the same 36-row fold basis, and Table 17 sets the two architectures’ knee points side by side.

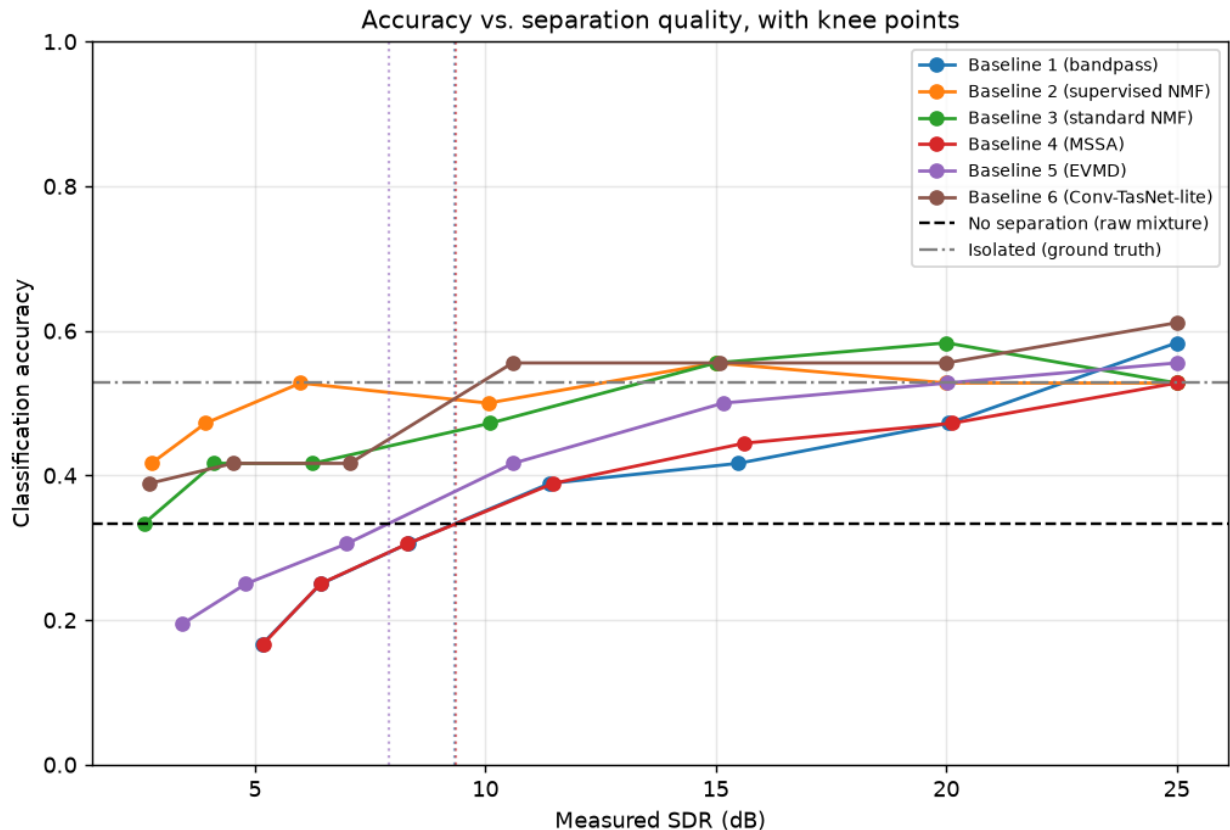


Figure 2: The same sweep classified with Architecture 2 (log-Mel + CNN). Its no-separation reference is 33.3% and its isolated accuracy on these 36 rows is 52.8%.

Table 17: Knee points under both architectures. “always above” means the curve never falls below that architecture’s no-separation reference at any measured SDR, so no knee exists in range. The detector’s agreement criterion is both knees clean and within 3 dB.

Baseline	Arch. 1 knee (dB)	Arch. 2 knee (dB)	Spread (dB)
Baseline 1 (bandpass)	20.1	9.3	10.7
Baseline 2 (supervised NMF)	always above	always above	—
Baseline 3 (standard NMF)	15.0	always above	—
Baseline 4 (MSSA)	15.6	9.4	6.3
Baseline 5 (EVMD)	18.2	7.9	10.3
Baseline 6 (Conv-TasNet-lite)	4.5	always above	—

Verdict: the knee *value* is not robust to the architecture; the grouping and the shape are. Under the study’s 3-dB operational agreement criterion — a working threshold chosen for this analysis, not a standard — no baseline agrees: the three crossings both architectures find are 6–11dB apart, every one lower under the CNN, and for the other three the CNN finds no crossing where the SVM finds one or, for supervised NMF, neither does. Two things account for most of that. The reference bar is lower for Architecture 2 (33.3% against 38.9%), so the same trajectory crosses it earlier; and the CNN’s trajectories are steeper and cleaner — every one of

its six is monotone or nearly so across the seven levels, rising from 17–42% to 53–61%, where the SVM’s are flat for the first three levels and only then climb. A crossing quoted to the decibel is therefore a property of the classifier that measured it, on this dataset at this sample size, and this report does not quote one.

What the two sets of trajectories share is the part that matters for the C2 question, and it is more than shape. Both split the six methods the same way: supervised NMF is “always above” the reference under both, Conv-TasNet is always above under the CNN and dips below by one row under the SVM, standard NMF is always above under the CNN and crosses at 15.0 dB under the SVM, and the three spectral-stripping methods — bandpass, MSSA, EVMD — cross late under both, at 15–20 dB for the SVM and 8–9 dB for the CNN. Under both, those three methods’ operating points (3.6–5.5 dB) lie below their crossing region, so their output classifies worse than the raw mixture. The robustness check therefore weakens the numerical claim of Section 8.8 and sharpens its qualitative one: what a downstream classifier responds to is not SDR as such but whether the separator preserves the spectral envelope, and the methods that do not would need to improve by several decibels before separating beat not separating.

How uncertain each crossing is, measured. Table 18 bootstraps the whole construction — trajectory, SDR axis and reference together, 2000 replicates, once over rows and once over the 24 leak groups the 36 rows belong to — and finds the crossing on every replicate with the same detector (`sdr_knee_bootstrap.py`). Two things the point estimates could not show. First, a crossing is not even guaranteed to exist: for Conv-TasNet under Architecture 1 a third of replicates find none, and for the mask methods under Architecture 2 most do not. Second, where it does exist its interval spans most of the measured range: every Architecture 1 interval reaches the 25 dB top of the grid and is reported as not identifiable there, and the tightest interval in the table (EVMD under the CNN, 3–15 dB) is still 12 dB wide. The cluster bootstrap moves nothing by more than about a decibel, so the rows’ dependence is not what limits the precision; $n=36$ is. What is stable across replicates is the ordering the text above relies on: under the CNN, Baselines 1, 4 and 5 cross in 90–98% of replicates and Baseline 2 in 15%, so “the strippers have a crossing and supervised NMF does not” survives resampling even though no individual crossing value does.

Table 18: Bootstrap uncertainty of the raw-mixture crossings, 2000 replicates, resampling the 36 rows (i.i.d.) or the 24 leak groups they fall into (cluster). Trajectory, its SDR axis and the no-separation reference are all recomputed on each replicate and the crossing found with the same detector. $P(\text{crossed})$ is the fraction of replicates with a clean crossing; the interval is the 2.5–97.5 percentile over those replicates; “n.i.” (not identifiable) means the percentile sits at the 25 dB top of the measured grid.

Arch.	Baseline	Rows ($n=36$)		Leak groups ($n=24$)	
		$P(\text{crossed})$	Median [95% CI]	$P(\text{crossed})$	Median [95% CI]
1	B1 bandpass	0.71	15.4 [7.3, n.i.]	0.75	15.6 [8.5, n.i.]
1	B2 supervised NMF	0.43	6.2 [3.4, 22.5]	0.39	5.7 [3.1, 18.7]
1	B3 standard NMF	0.90	15.0 [5.0, n.i.]	0.97	15.0 [6.0, 23.8]
1	B4 MSSA	0.72	15.4 [7.3, n.i.]	0.75	15.6 [7.3, n.i.]
1	B5 EVMD	0.85	18.0 [6.1, n.i.]	0.92	18.1 [7.3, n.i.]
1	B6 Conv-TasNet	0.63	18.4 [3.1, n.i.]	0.65	19.0 [2.8, n.i.]
2	B1 bandpass	0.95	9.5 [4.3, 21.7]	0.98	9.5 [3.6, 22.5]
2	B2 supervised NMF	0.15	3.9 [1.7, 22.5]	0.15	3.8 [1.7, n.i.]
2	B3 standard NMF	0.45	4.2 [1.7, 13.7]	0.45	4.6 [1.7, 14.2]
2	B4 MSSA	0.94	9.4 [4.3, 21.7]	0.94	9.3 [3.5, 23.4]
2	B5 EVMD	0.90	8.3 [3.2, 15.2]	0.93	8.2 [2.5, 15.2]
2	B6 Conv-TasNet	0.27	8.2 [2.9, 22.5]	0.25	8.5 [2.0, 23.3]

9 Computational Cost

Every result so far asks “how good” a method is. A separate, equally practical question — especially relevant given this project’s own related-work review of an FPGA-deployed lung classifier ([8], Table 27) — is “how expensive” each method is to actually run. This section measures that directly, using Section 2.9’s vocabulary (MACs, latency, the Pareto front), for all six separation baselines plus the Condition A classifier.

9.1 MACs per method

Table 19 reports the number of multiply-accumulate operations each method performs to process one real 60,000-sample (15 s) mixture, alongside each method’s persisted parameter count (zero for every zero-training method). Two precision levels are reported explicitly rather than presented as equally certain: *exact* counts come directly from a method’s own literal matrix-multiplication or convolution-layer shapes (or, for Conv-TasNet-lite, from forward hooks recording real output shapes during an actual forward pass, rather than hand-deriving stride/padding/dilation arithmetic by hand); *estimate* counts (MSSA’s SVD, EVMD’s ADMM loop) use standard textbook complexity formulas applied to this project’s real problem sizes, and are order-of-magnitude figures, not literal op-for-op tallies.

Table 19: MACs and parameters per method, one real 60,000-sample mixture/recording.

Method	Parameters	MACs/inference	Precision
Baseline 1 (bandpass)	0	2.4 M	exact
Condition A classifier (MFCC+SVM)	40 support vectors	8.1 M	exact
Baseline 2 (supervised NMF)	7,710 (frozen dictionary)	219 M	exact
Baseline 3 (standard NMF)	0 (transductive)	1.16 G	exact
Baseline 4 (MSSA)	0	1.20 G	estimate
Baseline 6 (Conv-TasNet-lite)	325,465	1.96 G	exact
Baseline 5 (EVMD)	0	5.22 G	estimate

Two findings are worth calling out specifically. First, Conv-TasNet-lite’s measured parameter count (325,465) independently reproduces the $\sim 325\text{K}$ figure Section 6.9 already reports from PyTorch’s own model summary — a genuine cross-check between two independently-computed numbers, not a coincidence of rounding. Second, Baseline 3 costs $\sim 5.3\times$ more MACs per mixture than Baseline 2 despite being the “simpler” ablation (no learned dictionary, Section 6.6): Baseline 3 is fully *transductive* (every one of its 160 multiplicative-update iterations updates both W and H fresh per mixture), while Baseline 2 only solves activations (60 iterations, H only) against an already-frozen dictionary. This gives the inductive-vs-transductive distinction already noted in Section 6.6 a concrete computational-cost number to go with the qualitative one.

9.2 Desktop latency under a uniform protocol

MACs are a theoretical proxy (Section 2.9); the actual number a deployment decision cares about is wall-clock time. Latency was measured under a *uniform protocol*, held identical across every method regardless of what else is running on the machine: one untimed warm-up call (so any one-time cache or JIT warm-up cost is excluded), then several timed repetitions on the same real 60,000-sample input, with the *median* (not mean, which a single slow repetition could dominate) and interquartile range reported for both wall-clock time and process CPU time. One-time setup cost — NMF dictionary fitting, Conv-TasNet-lite training — is fit once, untimed, before the timed region begins, since this section measures *inference* latency, matching the convention Yaqub et al. themselves use for their own model comparison (their Table 5 reports inference time, not training time).

Two runs, and why the first is retracted. The first measurement (2026-08-27) was taken on a shared machine under a 1-minute load average of 74.6 on 16 cores — this project’s own sweep generation plus other users’ jobs — and was reported with that caveat and the argument that a uniform protocol keeps the *ranking* meaningful. The re-measurement below, on the same machine idle (load average 0.6 at start, 1.6 at end), shows that argument was wrong: the loaded run inflated the CPU-bound methods by between $10\times$ (bandpass, $36 \rightarrow 3.6\text{ ms}$) and $340\times$ (standard NMF, $21,036 \rightarrow 61\text{ ms}$) while leaving the GPU-bound Conv-TasNet-lite nearly untouched ($23.9 \rightarrow 16.7\text{ ms}$), so it did not preserve the ranking either — it reordered the two fastest methods. Every latency-derived claim in this report is from the idle run. Table 20 gives it with the dispersion a reader needs to judge it; the measurement environment is recorded by the script itself (Table 21).

Table 20: Desktop inference latency per 15 s clip (60,000 samples), uniform protocol: one untimed warm-up, then n timed repetitions; median, IQR and p5–p95 of wall-clock time, median process CPU time, and the real-time factor (15s/median; above 1 is faster than real time). Per-call preprocessing inside each method (e.g. Baseline 2’s denoising bandpass and STFT) is included; one-time fitting and training are not. Machine idle: load average 0.6/16 cores.

Method	n	Wall median (ms)	IQR / p5–p95 (ms)	CPU median (ms)	Real-time factor
Baseline 1 (bandpass)	5	3.63	0.01 / 3.62–3.70	3.6	4,130×
Condition A classifier (MFCC+SVM)	5	3.69	0.02 / 3.67–3.74	63.6	4,060×
Baseline 6 (Conv-TasNet-lite), GPU	5	16.7	0.03 / 16.7–16.7	16.7	899×
Baseline 2 (supervised NMF)	5	17.2	2.2 / 17.1–22.2	107	873×
Baseline 3 (standard NMF)	5	61.4	0.7 / 60.6–61.6	367	244×
Baseline 4 (MSSA)	5	1,097	39 / 1,057–1,117	7,519	13.7×
Baseline 5 (EVMD)	3	16,812	49 / 16,794–16,882	16,812	0.89×

Table 21: Measurement environment for Table 20, recorded at run time (results/latency_environment.csv).

CPU	Intel Core i9-9900K @ 3.60 GHz, 16 logical CPUs
GPU	NVIDIA GeForce RTX 3060 (Conv-TasNet-lite only; all other methods CPU)
RAM	65.7 GB
OS / Python	Linux 6.8.0 (glibc 2.39) / 3.12.4
Libraries	numpy 2.4.6, scipy 1.17.1, librosa 0.11.0, scikit-learn 1.9.0, torch 2.3.1+cu121, mir_eval 0.8.2
Threads	torch 8; OMP_NUM_THREADS unset (BLAS default)
Batch	one clip per call, no batching
Load average	0.64 (1 min) at start, 1.6 at end

What the idle measurement says. Five of the seven methods run hundreds to thousands of times faster than real time on this workstation; MSSA runs at 14× real time and EVMD, at 0.89×, is the only method that cannot keep up with a live 15 s clip. The wall-clock ranking is now bandpass < classifier < Conv-TasNet-lite $\approx$ supervised NMF < standard NMF < MSSA < EVMD, and it is stable: IQRs are under 1% of the median for every method but supervised NMF (13%) and MSSA (4%).

MACs still do not predict latency, but the example has changed. Conv-TasNet-lite has 820× the bandpass filter’s MACs and runs 4.6× slower, not 820× — because it runs on the GPU. EVMD has 2.7× Conv-TasNet-lite’s MACs and runs 1,000× slower — because it is per-mode ADMM in NumPy on the CPU. Within one implementation family the two measures do agree: Baselines 2 and 3, both this project’s NMF code, have a CPU-time ratio of 367/107 $\approx$ 3.4× against a MACs ratio of 5.3×. The divergence *across* implementations is an implementation-efficiency and hardware effect, which is why Section 2.9 insisted on measuring both quantities; the loaded run had exaggerated it into Conv-TasNet-lite being the fastest method of all, which it is not.

Scope of these numbers. Every figure in this subsection is a desktop measurement: this workstation, running the Python implementations, idle. They rank the methods against each other on one machine; they are not a claim about what any of these methods costs on embedded hardware, where both the processor and the implementation differ. For Baseline 1 that claim now exists separately and is measured rather than extrapolated — see Section 10.3, which supersedes this subsection’s figure for that method alone.

9.3 The SDR-vs-compute plane

Bringing Section 7’s separation-quality numbers together with this section’s compute-cost numbers lets every baseline be placed on a single plane: separation quality (heart SDR, native additive subset) on one axis, compute cost — MACs, or measured latency — on the other. Two panels are used rather than one collapsed “compute” axis, precisely because Section 9.2 already found MACs and latency disagreeing substantially for at least one method; collapsing to a single axis would hide exactly the kind of divergence just demonstrated.

Section 2.9’s Pareto-dominance definition is applied directly: on a given axis, baseline A is *dominated* by baseline B if B has equal-or-better SDR *and* equal-or-lower compute cost, with at least one strict improvement — meaning A is never the right choice on that axis, regardless of how quality and cost are weighted against each other. Figure/Table generation for this plane (`sdr_compute_plane.py`) reads its SDR from the canonical row-level file (Table 10, pooled mean and 95% CI) and its latency from the idle-machine run of Table 20, so the plane is derived from the same artifacts as every other table rather than from numbers copied into the script.

Table 22: Pareto dominance per baseline, on the MACs axis and the latency axis separately.

Method	Heart SDR (dB)	Dominated on MACs?	Dominated on latency?
Baseline 1 (bandpass)	5.46 ± 2.66	No	No
Baseline 2 (supervised NMF)	2.58 ± 1.87	Yes	Yes
Baseline 3 (standard NMF)	3.35 ± 1.88	Yes	Yes
Baseline 4 (MSSA)	5.02 ± 2.91	Yes	Yes
Baseline 5 (EVMD)	3.61 ± 2.35	Yes	Yes
Baseline 6 (Conv-TasNet-lite)	5.02 ± 2.77	Yes	Yes

On the MACs plane, one method dominates outright. Bandpass filtering has both the highest heart SDR of the six baselines and, by a margin of nearly three orders of magnitude, the lowest MAC count (Table 19) — every other baseline is strictly worse on both axes simultaneously, and is therefore Pareto-dominated. If MACs were the only compute-cost figure available, the conclusion would be unambiguous: there is no reason, on this evidence, to prefer any learned or iterative separation method over the zero-training filter.

On the measured-latency plane the conclusion is the same, and an earlier revision of this report said otherwise. With the loaded-machine latencies, Conv-TasNet-lite (23.9 ms) appeared faster than bandpass (36.0 ms) and the two were reported as jointly non-dominated — a genuine trade-off. On the idle machine bandpass runs in 3.6 ms and Conv-TasNet-lite in 16.7 ms on a GPU, so bandpass has the higher SDR *and* the lower latency and every other baseline is dominated on both planes. The retracted finding was an artifact of measuring CPU-bound methods on a saturated CPU while the one GPU-bound method was unaffected; it is recorded here because it was published in an earlier revision and because it is a clean example of a plausible measurement that reversed a conclusion. What remains true is the weaker statement: Conv-TasNet-lite is the only method whose latency is within an order of magnitude of the filter’s despite three orders of magnitude more arithmetic, which is what a GPU buys.

10 Embedded Deployment: Baseline 1 on a Microcontroller

Every number in Section 9 is a desktop measurement or an analytic count. Neither answers the question a deployment decision actually asks: does this run on the hardware it is meant to run on? Section 11’s Edge-Enabled paper [8] targets a PYNQ-ZU FPGA at roughly 15 W; this section reports Baseline 1 ported to, and measured on, a considerably smaller target — an Arduino Nano 33 BLE Sense (nRF52840, 64 MHz Cortex-M4F, 256 kB RAM). The port lives in `firmware/`, with an ESP32-S3 target built from the same core but not yet exercised on silicon.

Baseline 1 is the natural first candidate, and not only because it is the simplest: Section 9.3 puts it alone on the Pareto front of both compute planes, so it is the method a deployment decision would consider first.

10.1 The one intentional divergence: causal filtering

The Python baseline calls `scipy.signal.sosfiltfilt`, which runs the cascade forwards and then backwards over the reversed signal. That cancels the filter’s phase response exactly, but it cannot begin until the *entire* recording exists. The firmware runs the equivalent of `sosfilt` instead: one causal pass, block-at-a-time, filter state carried across calls.

This is a forced choice, not a preference. A 15 s clip at 4000 Hz is 60,000 samples; as raw `int16` that is 120 kB, or 46% of the Nano 33’s total RAM before any processing begins, and `filtfilt` additionally needs a `float32` working copy of the same signal — 240 kB more, on a part with 256 kB in total. It does not fit. Even where it would fit, it forfeits the point: no output could be produced until the recording ended, which is buffered batch processing wearing a microcontroller.

Two costs follow, both real. The output carries the filter’s phase response and a non-zero group delay, so waveform-shape comparisons against the Python baseline are not like-for-like. And `sosfiltfilt` applies the same cascade twice, so its stopbands are twice as deep in dB — one pass of the order-4 design is a genuinely weaker filter than what Section 6.4 reports.

Measured, not assumed. Running both filters over the same 145 `Mix.csv` rows with identical coefficients and identical BSS Eval metrics — changing only `sosfiltfilt` to `sosfilt` — gives the mean differences in Table 23.

Table 23: Causal minus zero-phase, mean over 145 rows. Only the filter direction changes.

Source	Δ SDR	Δ SIR	Δ SAR
Heart	+0.18 dB	−0.13 dB	+5.47 dB
Lung	−0.04 dB	−0.50 dB	+7.30 dB

SDR is unchanged within ± 0.2 dB — on this dataset the causal filter costs essentially nothing on the headline metric, better than the theory above would suggest. SIR drops slightly, exactly as the halved effective stopband depth predicts, since rejecting the other source is what stopband depth buys. **The large SAR gain is a metric artifact and must not be reported as the firmware separating more cleanly:** `mir_eval`’s projection fits a *causal* 512-tap distortion filter, so a causal IIR’s phase response lies inside the allowed-distortion subspace and is absorbed, while a zero-phase filter’s acausal response does not fit and is partly scored as artifacts.

10.2 Verifying a port that has no reference implementation on the target

A build that links is not a build that computes the right thing, and the board runs a different kernel from the host: CMSIS-DSP’s `arm_biquad_cascade_df2T_f32` on the Nano 33, ESP-DSP’s `dsps_biquad_f32` on the ESP32-S3, against portable C on the desktop. The three sit behind one API so the same source builds for all three, but the vendor kernels are third-party code, and CMSIS writes the recursion with the opposite sign convention for a_1/a_2 from scipy’s — a detail the initialisation has to correct for and which no compiler would catch.

The port is therefore verified in four stages, three of which need no hardware at all:

1. **Host, portable C vs scipy.** The C core filters the same golden vectors scipy produced and the deviation is compared against the `float32` quantisation floor. Also checked here: identical output for block sizes 1, 7, 64 and 333, which is what proves the streaming state is actually carried across calls rather than silently reset.
2. **Host, wire protocol.** An emulated board over a pseudo-terminal exercises the serial framing in both directions, confirming a faithful stream is accepted and a corrupted one rejected rather than silently scored.
3. **On silicon, vendor kernel.** The `*-selftest` firmware replays a 1024-sample reference held in flash through the vendor kernel and prints the deviation over serial. This must be run *before* any measurement: if the kernel is wrong, every subsequent number is wrong in a way that looks like a filter-design problem.
4. **On silicon, full clips.** The `*-bench` firmware replays real HLS-CMDS mixtures compiled into flash and streams the filtered output back for scoring.

The bench firmware deliberately has *no microphone*. It filters the same `int16` samples the Python baseline filters — bit-identical, since the $1/32768$ scaling reproduces `librosa.load()` exactly — so the board’s output is diffable against scipy sample by sample and scoreable with the same BSS Eval metrics as the rest of this report. A microphone would introduce a transducer, a resampling step and a gain to calibrate away, none of which the question being asked here needs. Microphone firmwares (PDM on the Nano 33, I2S on the ESP32-S3) exist for a future real device but are not in the measurement path.

Only additive mixtures are embedded. Section 5 found 109 of 145 rows whose mixture is not $a \cdot (\text{heart} + \text{lung})$ of its named sources, and an SDR scored against those references measures the dataset’s defect rather than the separator. The first hardware run embedded M0001–M0003, all three non-additive, which left its SDR column unquotable; selection now draws from the 36 valid rows.

10.3 Results on silicon

Table 24 reports the first hardware run: three additive mixtures replayed from flash on a Nano 33 BLE Sense.

Table 24: Nano 33 BLE Sense, three additive mixtures replayed from flash.

Clip	board – scipy , $\times$ float32 floor		SDR (dB), board = scipy		$\mu\text{s/sample}$
	heart	lung	heart	lung	
M0087	2.04	0.89	4.918	3.686	3.474
M0111	1.26	0.90	18.91	–12.43	3.488
M0112	0.82	0.71	4.556	1.197	3.431

The port is faithful. Every SDR agrees with the scipy reference to the last printed digit, on all six clip/source measurements. These are not two independent estimates of the same quantity that happen to agree — they are the same computation performed on two different machines, and the agreement is the check.

Timing: 3.47 μs per sample, 72 $\times$ faster than real time. A 15 s clip filters in 209 ms of compute on a 64 MHz Cortex-M4F, measured around the `int16` scaling and both band cascades with serial I/O excluded. That headroom is the deployment-relevant result: this band split can run as a live stream on a part costing a few dollars, with two orders of magnitude to spare. For Baseline 1 this figure supersedes Section 9.2’s desktop measurement, which timed a different machine doing a different amount of work. Nothing in the firmware’s RAM usage scales with recording length — the filters are streaming and the clips live in flash — so a 15-second clip and an hour-long stream cost the same few hundred bytes of filter state.

10.4 A plausible measurement that was wrong by 18 $\times$

That timing figure was originally 62.4 μs per sample — 4.0 $\times$ real time. Nothing about it looked wrong. It was a sensible number for a 64 MHz part, it was consistent across all three clips to three significant figures, and it came from a firmware whose numerical output had already been verified against scipy twice. It was also 18 $\times$ too slow, and the reason is worth recording because nothing in the measurement itself could have revealed it.

The discrepancy surfaced only from the resource model of Section 10.8, which predicted roughly 400 $\times$ real time for this method. A model being optimistic by 5 $\times$ is ordinary; being optimistic by 100 $\times$ is a claim that something is wrong somewhere, and the model was specific enough to be worth trusting over the intuition that 62 μs “seemed fine”.

Reading the actual compiler invocation — rather than the build configuration, which is where the intent lives but not the outcome — showed the cause. The Arduino-mbed builder appends `-nostdlib -mfloat-abi=soft` *after* the framework’s own earlier `-mfloat-abi=softfp`, and GCC honours the last occurrence of that flag. Every floating-point operation in the firmware was therefore compiling to `__aeabi_*` software-float library calls. Disassembling both the project’s own filter object and CMSIS-DSP’s biquad kernel confirmed it directly: zero VFP instructions in either, on a Cortex-M4F whose floating-point unit is the entire reason for the suffix.

Adding the correct flag to the build configuration did not fix it, because those flags are emitted *before* the builder’s trailing group and so still lost the last-occurrence contest — a detail visible only by re-reading the compile line after the attempted fix, and easy to miss by assuming an override had taken effect. PlatformIO’s `build_unflags` removes the offending flag outright. After the change: VFP instructions present in both objects, no software-float calls, 1,248 fewer bytes of flash, and the re-measurement in Table 24.

Two things generalise from this. First, a performance figure that looks plausible has not been validated by looking plausible; here the only thing that caught it was an independent model dis-

agreeing loudly enough to be worth chasing. Second, the self-test of Section 10.2 passed before and after — correctly, since software floating point computes the right answer, only slowly. Numerical verification and performance verification are separate obligations, and passing the first says nothing about the second.

10.5 A tolerance that was calibrated on the wrong signal

The first run on additive clips *failed* its accuracy check, at 1.045×10^{-5} against an absolute tolerance of 10^{-5} — on a board whose on-silicon self-test had just passed at 5.3×10^{-8} and whose SDR agreed with *scipy* to four significant figures. The tolerance was wrong, not the kernel, and the reason is worth recording because it is a general trap.

The board computes in `float32` while the reference is `float64`, so the gap between them is not a constant: it scales with signal amplitude, because an IIR accumulates rounding through its own recursive state. The 10^{-5} figure had been calibrated on M0001–M0003, which peak near 0.05. The 36 additive rows are peak-normalised to full scale — that normalisation is precisely the clipping fingerprint that identifies them (Section 5) — so they are roughly $20\times$ louder, and the same correct arithmetic lands roughly $20\times$ further from the reference.

The decisive check is to run *scipy itself* in `float32` on the same clip: on M0112 that deviates by 1.114×10^{-5} , which is *larger* than the board’s 1.045×10^{-5} . The board was not merely acceptable; it was more accurate than the reference implementation would have been at the same precision.

The criterion is now measured rather than assumed: the `float32` floor is computed per clip by filtering the same samples with *scipy* in `float32`, and the board is held to $4\times$ whatever that produces. Observed ratios on real hardware are 0.71–2.04 (Table 24). For calibration, deliberately injected faults score far higher: a 0.01% gain error gives 14.3, a single dropped sample 15,691, and a sign inversion 296,503 — so the threshold sits in a wide gap, with roughly $7\times$ of margin above the real board and two further orders of magnitude below the smallest fault tested.

10.6 Server against microcontroller, on identical work

The natural question once a method runs in both places is what the move costs. Answering it needs more care than dividing Section 9.2’s desktop figure by the board’s, for two reasons that both flatter the board if left alone.

The two platforms were not doing the same work. Section 9.2 times `bandpass_separate`, which calls `sosfiltfilt`: forwards then backwards, two passes per band. The firmware makes one causal pass (Section 10.1). The desktop figure was therefore for roughly twice the arithmetic, and `server_vs_board.py` re-times the workstation doing exactly what the board does rather than halving the existing number and assuming the constant factors cancel. Measured directly, the zero-phase version costs $4.8\times$ the causal one, not the $2\times$ its two passes alone would suggest — `sosfiltfilt` also pads the signal, reverses it twice, and solves for initial conditions per section. Halving the existing figure would therefore have been wrong by more than a factor of two.

A raw ratio conflates two different claims. 3.6 GHz against 64 MHz is a $56\times$ clock disadvantage before architecture enters the picture. “This processor is faster” and “this processor does more per cycle” are separate statements, and only the second says anything about whether the port was a good idea. Cycles per sample separates them.

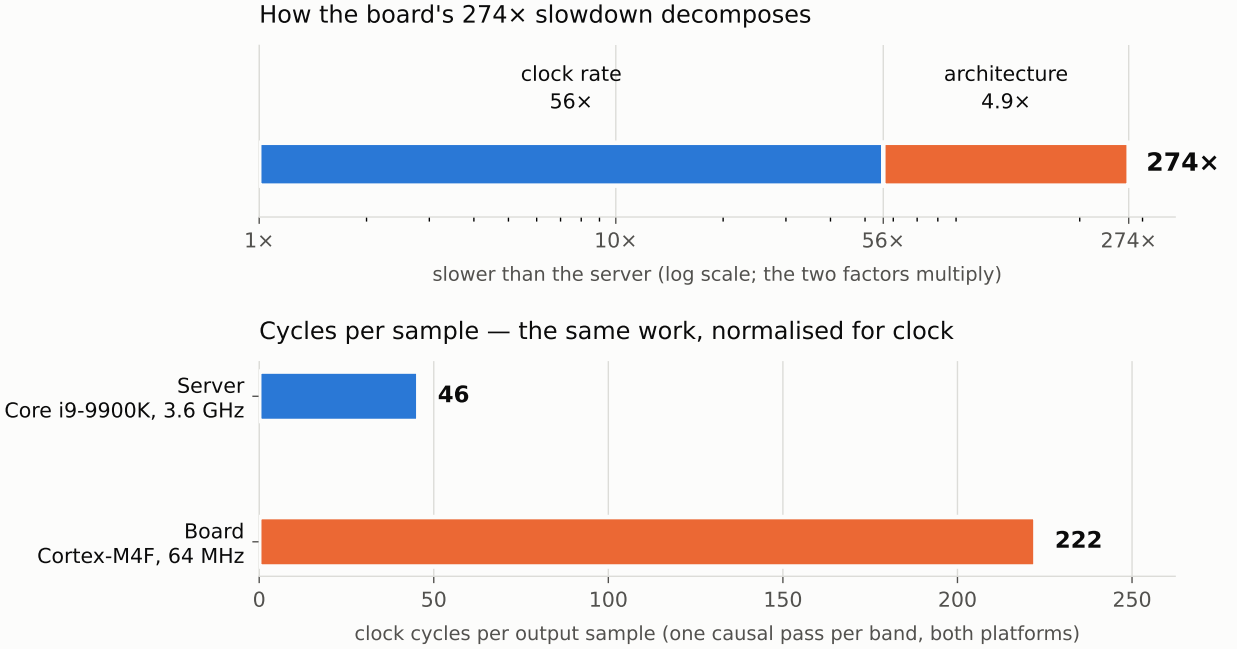


Figure 3: The 276× throughput gap decomposed. Because the two factors multiply rather than add, the top panel places them on a log axis, where the segment lengths are exactly $\log 56$ and $\log 4.9$. The lower panel is the measurement underneath: the same causal band split, normalised for clock rate, where the gap is a factor of five rather than a factor of 276.

Table 25: Identical workload — one causal pass per band over the same 60,000-sample mixture. Server figures are the median of five independent runs (spread under 1%) at load average 0.1; board figures measured on silicon.

Platform	ms/clip	µs/sample	cycles/sample	× real-time
Server, i9-9900K @ 3.6 GHz	0.75	0.0126	45.2	19,900
Board, Cortex-M4F @ 64 MHz	208.5	3.474	222.3	72
Board, before the FPU fix	3,747	62.45	3,997	4.0

Most of the gap is clock rate, but not all of it. The board is 275× slower per sample against a 56× clock disadvantage, so per clock cycle it spends 4.9× what the workstation does on the same filtering. Decomposing the 275× that way is the useful move: roughly a fifth of it is architecture and the rest is simply being clocked slower.

Both parts are explicable at the level of the inner loop. Each output sample needs eight biquad sections (four per band, two bands), so the workstation spends about 5.7 cycles per section and the board about 28. A direct-form-II transposed biquad is five multiply-accumulates with a serial dependency between consecutive samples: the 5.7 is close to what a dependency-limited chain costs on an out-of-order core, and the 28 is the expected cost of the same five operations as VFP instructions on an in-order Cortex-M4F, plus loop overhead. Neither number is anomalous, which is the point — the comparison is measuring the two architectures rather than an artefact.

This is the substantive argument for the port, stated at its real strength: a microcontroller gives up about a factor of five in per-cycle efficiency here, not the factor of 275 the raw timing

suggests, and $72\times$ real-time headroom makes that affordable. The serial structure of an IIR cascade is what keeps the architectural penalty this small — issue width and out-of-order execution have a dependency chain to fight. A method with exploitable instruction-level parallelism, or one whose working set fits a desktop cache but not 256 kB of SRAM, would show a much worse ratio; Section 10.8 covers what happens to those methods here for a more basic reason.

The last row is the same silicon before the build fix of Section 10.4, at 3,997 cycles per sample — an $88\times$ per-cycle penalty against the workstation rather than $4.9\times$. It is kept because it is what this comparison would have concluded from a measurement that looked entirely reasonable at the time: not “the toolchain is misconfigured” but “microcontrollers are getting on for two orders of magnitude less efficient per cycle than desktops, which is simply what embedded DSP costs”. That conclusion would have been wrong, quotable, and very hard to dislodge later.

The server side of this table needed the same correction for the same reason. Its first version timed a function that designed the filter coefficients inside the timed call — work the firmware never does, since its coefficients are compile-time constants in flash — and so charged the workstation $2.8\times$ its real cost, which would have reported the architectural penalty as $1.7\times$ rather than $4.9\times$. That figure was also stable across repetitions and plausible on its face. Two independent instances in one comparison is a reasonable argument for treating “the number looks sensible” as carrying no evidential weight at all.

10.7 Energy, where the answer is not yet known

Throughput is the wrong axis for a battery-powered device, which is judged on joules. The two need not agree: a processor $276\times$ slower that draws $1000\times$ less power still wins on energy. Half of that comparison can be measured here and half cannot, and the half that cannot turns out to be decisive.

The server side is measured, with no instrumentation. Intel CPUs expose a package energy counter through RAPL as an ordinary file in `/sys`, in microjoules, so sampling it either side of a known workload gives real energy rather than a TDP estimate. Idle package power is 28.0 W; under the board’s exact workload it is 46.7 W, so the computation itself draws 18.7 W above idle. At 0.75 ms per clip that is **14.3 mJ** marginal, or 35.8 mJ counting the whole package — median of five runs, spread 2%. Both are reported because the right one depends on the question: marginal if the server exists anyway and one more clip is the question, full package if a dedicated device is being compared against a dedicated machine.

The board side is not measured, and cannot be from here. The Nano 33 BLE Sense has no onboard current sense, so the firmware cannot report its own consumption and nothing on the server can infer it. What is reported instead is the *break-even*: since the board’s time per clip is known precisely (208.5 ms), energy is that time multiplied by the one missing number, and parity arrives at **69 mW** against the server’s marginal cost, or 172 mW against its full package.

That number is why no datasheet figure is substituted. A development board with USB, regulators, onboard sensors and an LED awake can plausibly sit on either side of 69 mW, so the conclusion flips inside the range a reasonable assumption would cover. Where a comparison is insensitive to an assumption, assuming is fine; this one inverts, and an inverted conclusion asserted from a datasheet would be worse than an acknowledged gap. An inline USB power meter during a ***-bench** run would settle it to the order of magnitude the argument needs.

Two things are worth noting about what such a measurement would show. If the board does come in under 69 mW, then the platform that is $276\times$ slower is simultaneously cheaper per clip, and the deployment argument reverses relative to Table 25 — which is the result this section exists to flag as plausible rather than to claim. And the same measurement would price the build flag of

Section 10.4: the pre-fix binary ran $18\times$ longer for identical output, so at comparable power draw it also cost $18\times$ the energy, which on a battery is the difference between a day and a fortnight.

10.8 Why the other five baselines do not run here

Porting Baseline 1 raises the obvious question: is there anything *better* that would also fit? Attempting five more ports to find out would be expensive, so `src/mcu_feasibility.py` models each method’s peak working memory, parameter storage and compute time against both targets, deriving every figure from the method’s own imported constants.

The discriminator is not arithmetic. A method is *streaming* if it produces output from a bounded window of input while carrying only fixed state between calls, and *batch* if the whole recording must be resident before any output exists. Baseline 1 is streaming: 16 floats of biquad state, identical whether the recording is 15 seconds or an hour. That property, not the MAC count, decides most of Table 26.

Table 26: Peak working RAM against the 211 kB left on a Nano 33 BLE Sense after the Baseline 1 firmware’s own footprint. All figures at `float32`.

Method	Streaming	Peak RAM	Verdict
Baseline 1 (bandpass)	yes	832 B	fits ($0.004\times$)
Baseline 2 (supervised NMF)	yes	9.1 kB	fits ($0.04\times$)
Baseline 3 (standard NMF)	no	1.0 MB	$4.7\times$ over
Baseline 4 (MSSA)	no	36 MB	$167\times$ over
Baseline 5 (EVMD)	no	15.4 MB	$71\times$ over
Baseline 6 (Conv-TasNet-lite)	no	9.2 MB	$43\times$ over

Baselines 4 and 5 are batch by construction, not by implementation. SSA embeds the signal into an $L \times K$ trajectory matrix ($L=50$, $K=59,951$), takes its SVD, and reconstructs each component by diagonal averaging back across the full length; every one of those structures is $O(LN)$, and the reconstruction references the whole signal. VMD’s ADMM iterates in the frequency domain over the mirror-padded signal, holding all K modes and their spectra live across every iteration. Neither has a streaming formulation to be written more carefully — the algorithms themselves are defined over the complete recording.

Baseline 6 is batch only as written. It encodes the whole clip and runs the TCN over all 6,000 resulting frames at once, so its 9.2 MB is activations rather than weights — the parameters are 318 kB at `int8` and fit flash comfortably. A chunked, causal Conv-TasNet is a known variant and would make activations $O(\text{chunk})$; that is a redesign of the model, not a deployment step, and it is the honest description of what stands between this method and the board.

Quantisation does not rescue any of them. Moving from `float32` to `int8` divides memory by four, which leaves Baseline 3 at $1.2\times$ over, Baseline 6 at $10.7\times$, Baseline 5 at $17.8\times$ and Baseline 4 at $41.6\times$. None fits. More fundamentally, for most of them `int8` is not an applicable representation at all: Baseline 5’s ADMM convergence tolerance is 10^{-6} , four orders of magnitude below `int8`’s resolution of roughly 4×10^{-3} , so the solver could not detect its own convergence; Baseline 4 selects components by a 2% eigenvalue-contribution threshold across singular values spanning many orders of magnitude, which 256 levels cannot represent; and the NMF multiplicative updates carry a 10^{-12} epsilon floor. Only Baseline 6 is a natural quantisation target — neural post-training quantisation being mature and well tooled — and for it quantisation is the *last* step after the streaming redesign, not an alternative to it.

What this means for the deployment question. On the native additive subset (Table 10), the only method that beats Baseline 1 on either source is Baseline 4 on lung, by 1.34dB — and Baseline 4 is the least feasible entry in the table. Every method that could run here scores worse than Baseline 1 on both sources. The finding is therefore a negative one, and worth stating plainly: on this dataset, with these six methods, the cheapest method is also the best-performing one, which is why Section 9.3 already placed it on the Pareto front of both compute planes. Baseline 2 remains the one non-trivial port that would fit, and its value would be demonstrating that the firmware’s verification chain generalises to a fundamentally different algorithm — FFT and iterative matrix updates rather than a biquad cascade — not improving separation quality.

One caveat on the timing column. The model’s compute estimates assume one MAC per cycle with operands in registers. Calibrated against the one method now measured on silicon, it is optimistic by $5.6\times$ ($400\times$ predicted, $72\times$ measured). Applying that factor to Baseline 2 would put it at $0.8\times$ real time rather than $4.4\times$ — so its real-time margin is genuinely uncertain, and would need measuring rather than asserting. The memory columns, which decide every infeasible row, are unaffected.

11 Related Work

Table 27 summarizes six papers reviewed against this work, organized by what each one measures. Full citation details for all six are now filled in below, sourced from the Notion Reading List; five of the six have since been independently cross-checked against their primary-source PDFs rather than relied on second-hand summaries ([5], a Ph.D. dissertation, remains the one exception — flagged in the bibliography).

Table 27: Related work, organized by what is measured.

Paper	Dataset	Separation method	Separation metrics	Downstream task	Split
HLS-CMDS descriptor [1]	de- — (source)	—	—	—	—
AI-Driven Car-diorespiratory Signal Processing [5]	HLS-CMDS	PL-NMF + LLM-guided fundamental frequency (“LingoNMF”)	not reported (paired t -test on unspecified proxy)	clustering + anomaly detection, not accuracy	not stated
Spectrotemporal DL for Heart Sound Classification [6]	PhysioNet 2016 (training) + HLS-CMDS (external validation, Sec. 5 of the paper)	bandpass filter (their Experiment 4 stress test: heart sounds computationally separated from mixed HLS-CMDS heart-lung recordings)	none (no SDR/SIR/SAR — only downstream accuracy is reported)	4-class PCG classification (Normal/Murmur/Extra Sound/Rhythm Disorder); accuracy collapses 89.0%→41.0% (Exp. 3 clean vs. Exp. 4 separated) — see Sec. 11.1 below, this is the motivating result for this project’s whole re-search question	not stated
NMF-Enhanced Log-Mel + CRNN [7]	ICBHI 2017 + Fraiwan CWLS	NMF-based enhancement	none (no SNR/SDR/correlation)	multi-class cardiorespiratory disease classification	recording-level (leakage flagged by authors)
Edge-Enabled Portable Lung-Sound Classifier [8]	HLS-CMDS	proposed lung isolation from mixtures	none	lung sound classification	not stated
Performance Measurement in Blind ASS [2]	— (methods)	—	defines SDR/SIR/SAR	n/a	n/a
Cardiorespiratory Separation via SSA [9]	not stated	two-stage Singular Spectrum Analysis	SDR, STOI, correlation vs. 5 baselines	none	n/a

11.1 Yaqub et al.: the motivating result, not an unrelated classification paper

An earlier draft of this table filed [6] under “PhysioNet 2016, no separation, binary normal/abnormal classification.” That was wrong on every count that matters, confirmed now by reading the primary source directly rather than a second-hand summary. Yaqub et al. train a ResNet-18 PCG classifier on PhysioNet 2016, then use HLS-CMDS purely for *external validation* across four experiments: (1) binary fine-tuning, 88.0% accuracy; (2) 4-class fine-tuning, 86.0%; (3) full retraining from Im-

ageNet weights on HLS-CMDS’s own 4-class labels, 89.0% (their best, most balanced result); and (4) a stress test that re-evaluates the Experiment-3 model on heart sounds *computationally separated from HLS-CMDS’s mixed heart-lung recordings using a standard bandpass filter* — where accuracy collapses to 41.0% (Table 9 of [6]: $n=5365$, weighted F1 falls from 0.89 to 0.39, every class’s precision and recall degrade, not just one).

This is not a tangential result. It is the single empirical finding this project’s entire research question exists to explain: separating a mixed cardiorespiratory recording before classifying it costs something, and Yaqub et al. are the first to measure that cost directly, end to end, on this project’s own dataset. Their own Discussion (Sec. 6.2) attributes the collapse specifically to “the subtle distortions and potential information loss introduced by the standard bandpass filter used for source separation” and its “inherent limitation... to perfectly separate spectrally overlapping signals” — which is exactly the same class of method as this report’s own Baseline 1 (Section 6.4), and exactly the SIR-vs-SAR tradeoff this project’s own bandpass results demonstrate directly on the separation-quality side (real SIR gain, real SAR cost). Yaqub et al. show what that tradeoff costs a downstream classifier; this project’s separation-quality metrics are the mechanism that predicts when it will. Their 89%→41% is one point on the curve this project’s Condition A/B comparison (Section 13) is built to trace out continuously against SDR, not a two-point anecdote to reproduce once.

11.2 Metrics caveats from the BSS Eval literature

Two points from [2] directly shaped the design in Section 6.1–6.4: (1) SAR is sensitive to distortion that a downstream classifier’s accuracy may not register at all (a classifier can stay correct despite real signal artifacts, if the distortion doesn’t cross a decision boundary), which motivates reporting SDR/SIR/SAR rather than accuracy alone; and (2) SDR is not a perceptual measure and is gameable — a mildly low-pass-filtered estimate can drive SDR toward $+\infty$ under a time-varying filter without the estimate actually improving — which is why Tables 9 and 10 are reported with all three metrics rather than SDR in isolation. The same source notes that HLS-CMDS mixtures are heart+lung only, with no independent sensor-noise reference, so the noise term in a full images-based BSS Eval decomposition is effectively zero here; SAR in this report reflects algorithmic artifact, not resilience to real acquisition noise.

One further cross-check surfaced while reviewing [8]: that paper’s stated HLS-CMDS lung class counts (Normal 28, Wheezing 28, Pleural Rub 25, Rhonchi 23, Fine Crackles 22, Coarse Crackles 19) match the *total-in-Mix.zip* counts already encoded in this repository’s `load_dataset.py` (its `PAPER_LUNG_COUNTS mix_zip` column, validated against the live CSVs), not the smaller isolated-LS.csv count (5–12 per class). The two counts are easy to conflate and any report drawing on this dataset should be explicit about which one it cites.

12 Discussion

Of the six papers reviewed, exactly one — [8] — evaluates on HLS-CMDS *and* classifies mixture-separated audio, but reports zero separation-quality metrics and, per the summary reviewed, does not compare against classifying the isolated ground truth directly. The SSA paper [9] reports proper separation metrics but no classification task at all; the NMF+CRNN paper [7] reports classification accuracy with real numbers but on a different dataset, with no separation metrics, and its own authors flag a recording-level (leakage-prone) split as a limitation. No paper reviewed reports separation quality, separated-audio classification accuracy, and isolated-ground-truth classification

accuracy together, under a split verified not to leak recordings between the pool used to fit anything and the pool used to evaluate it.

That fourth condition is not a hypothetical concern for this dataset specifically: Section 4’s audit found that a third of the isolated recordings are literal duplicates of recordings embedded in the mixtures, and that mix rows chain together into leak groups as large as 32 of 145 rows. A recording-level split — the exact limitation [7] flags in its own results — would leak ground truth into training on this dataset in a very concrete way.

The Baseline 1 result (Section 7) demonstrates the SIR/artifact trade-off Section 11.2 predicts on real data rather than asserting it from theory: a real, if crude, separation method’s SDR gain over doing nothing is much smaller than its SIR gain alone would suggest, because SAR moves sharply in the opposite direction. This is the concrete case for reporting classification accuracy alongside SDR/SIR/SAR in the planned next phase (see `PROTOCOL.md`) rather than either metric family in isolation: a separation method that trades SIR for SAR in this way may or may not hurt a downstream classifier, and that is an empirical question this pipeline is built to answer, not one this report answers on its own.

Baseline 2’s comparison to Baseline 1 raises a second, more specific concern. On the full row set the two methods were close to tied; on the 36-row subset where genuine separation is actually possible (Section 5), leakage-safe supervised NMF is clearly *worse* than the zero-training filter, not comparable to it. Two non-exclusive explanations follow from how this pipeline is built. First, closing the leakage gap (Section 4) removes exactly the kind of inflation a naïve reproduction would have kept — per this project’s own working assumption, a lower, leakage-safe number is the *correct* one, not evidence the method itself is broken. Second, the gap is far larger and more consistent on the valid subset than on the full set, which is the opposite of what pure leakage inflation alone would predict (leakage should inflate NMF’s fitted-dictionary case regardless of row validity); this instead points toward the fixed dictionary rank ($K_i=10$, $K_r=20$) or the fixed iteration counts (Section 6.5) not transferring well to this dataset’s real acoustic mixtures, as opposed to the synthetically-summed or curated mixtures a published method may have been tuned against. A third, colder explanation was raised informally while Baseline 2 was being built: an *oracle* mask built from one mixture row’s own true heart/lung spectrograms — cheating on purpose, purely to bound the method’s ceiling — appeared to beat the no-separation reference only modestly, which would have meant the dataset’s real mixture recordings impose a hard ceiling on mask-based separation that no amount of NMF sophistication fixes. That probe is now a committed script (`oracle_mask_ceiling.py`) run over all 36 valid rows on Baseline 2’s own STFT grid, and it does not survive the re-check. The ideal ratio mask — exactly Baseline 2’s mask form with the true magnitudes substituted — scores 11.5 ± 1.9 dB heart SDR and 9.2 ± 1.7 dB lung (95% CI over rows), against 2.5 and -2.0 dB for the raw mixture and 5.5 and 4.1 dB for bandpass; the ideal Wiener filter reaches 12.4 and 10.6 dB. Baseline 2 therefore sits 7.8 dB (heart) and 9.7 dB (lung) below the ceiling of its own method family, and the shortfall is the dictionaries’ and activations’, not the dataset’s. The one-row note was almost certainly made on a non-additive row, where Section 5 shows every method — an oracle included — inherits the unrelated mixture’s energy as unremovable distortion. The check does establish a ceiling of a different kind: a magnitude mask on a 512-point STFT tops out near 12 dB on these recordings, which is below the 15–20 dB region where Section 8.8’s first classifier starts to prefer the output of a spectral-stripping separator. Whether a finer time-frequency grid or a time-domain separator can pass that cap is not tested here.

13 Limitations and Future Work

- **The descriptor’s stated sample rate is unreconciled.** The released dataset measures 4000 Hz throughout (Section 3.1) against the 22,050 Hz its descriptor paper states. Provenance itself is settled — the copy audited here is byte-identical to the Mendeley release — so the open question is narrower than it first appeared: whether the descriptor misstates the rate, or whether the figure was mis-transcribed downstream of it. Resolving that against the descriptor’s own text, with a page reference, is outstanding, and it determines how strongly the companion dataset-audit comment can phrase the recommendation.
- **The trajectories are complete but their vertical resolution is 36 rows, and a crossing is a point estimate from seven sampled levels.** Section 8.8’s trajectories and Section 8.9’s repeat of them are now full six-baseline results, and both say the same thing qualitatively. But every point on them is an accuracy over 36 recordings — one row is 2.8 points, and a binomial 95% interval near 40% is about ± 16 points — which is why the crossing *values* move by 6–11 dB between two classifiers, why their bootstrap intervals reach the top of the measured grid (Table 18), and why this report quotes a crossing region under a stated interpolation protocol, not a threshold. The 36 rows are also not 36 independent units: they fall into 24 leak groups (one of 5, one of 3, six of 2, sixteen singletons), so the effective sample size lies between 36 (no within-group correlation) and 17.5 (Kish, perfect correlation); the empirical within-group correlation of per-row correctness is 0.24 on isolated audio and near zero on separated audio, which puts it near 29. The leak-group bootstrap of Table 18 already accounts for this and widens nothing by more than a decibel. Tightening that needs more valid rows than HLS-CMDS has, which is the synthetic substrate’s job (Section 6.3); running the sweep and both classifiers over its 1500 rows is the direct next step.
- **Architecture 2 was tuned once, by hand.** Section 8.9’s CNN went from 30.0% to 62.0% through feature standardisation and a longer mini-batch schedule, with three configurations tried and the selection made on training-set fit; that is a small, disclosed search, but it is a search, and the adopted configuration was also the one with the best test accuracy at the time. The level normalisation added afterwards left its accuracy unchanged and narrowed its CI to ± 3.9 points. A nested selection, or simply a second seed, would say how much of the 62.0% is the configuration and how much is the fold draw — neither has been run.
- **The level confound was caught by its symptom, not by design.** Section 8.7’s first run was wrong in a way that produced a plausible table, and it was found only because three unrelated methods returned identical predictions. The classifiers are now level-invariant, but nothing yet checks that separated inputs fall inside the region where the SVM’s kernel is informative; a standardised-distance report per condition would make the next such defect visible before it is published.
- **Six separation methods evaluated; none dominates cleanly.** Baselines 1, 4, and 5 are zero-training sanity floors by design; Baselines 2, 3, and 6 involve some form of fitting. Section 7.3 shows Baseline 6 (Conv-TasNet-lite) scores best on synthetic-substrate SDR for both sources and on native heart SDR, but loses to Baseline 4 (MSSA) on native lung SDR by more than either method’s CI half-width — so this pipeline still does not have a single method that demonstrably and consistently improves on the strongest zero-training baseline across both sources and both substrates. LLM-guided NMF (“LingoNMF”), the one reviewed method not yet reproduced (Table 27), remains an open candidate.

- **Baseline 6 is an undertrained first neural baseline, not a tuned model.** Table 12 shows every fold exhausts its 25-epoch budget without the early-stopping criterion ever firing, on a pool of ~ 32 training recordings per class per fold — consistent with a compute-budget-limited result rather than a converged one (Section 6.9). Its results should be read as infrastructure validation, not as this project’s best achievable neural result; a longer training budget and/or more aggressive on-the-fly augmentation are the natural next steps before treating Baseline 6’s ranking against Baselines 1–5 as final.
- **Baseline 5’s synthetic-substrate column is a subsample, not the full set.** The full $K=2..10$ VMD sweep at the full synthetic $n \approx 1500$ was measured at ~ 15 s/mixture (Section 6.8), extrapolating to multi-hour runtime; its synthetic column in Table 11 therefore runs on a stratified 75-row subsample, giving it a visibly wider CI than the other five methods’ 1500-row columns. This is a disclosed scope reduction, not a like-for-like comparison, and should not be read as Baseline 5 being noisier in an absolute sense.
- **The synthetic set’s gain range is fitted from a small sample.** `synthetic_mix.py`’s gain distribution (Section 6.3) is fitted from the 36 native additive rows rather than chosen arbitrarily, which is a strength of the construction and equally a dependency of it: the synthetic substrate inherits whatever is idiosyncratic about those 36 rows’ own mixing gains ($3.5\text{--}135\times$), and $n=36$ is a small sample to fit a range from. A sensitivity check — rebuilding the synthetic set under a deliberately wider and a deliberately narrower gain range, then re-running one baseline — would bound how much Table 11 depends on that fit.
- **Baseline 2 hyperparameters: verified against the primary source, with two corrections.** An earlier revision listed the NMF dictionary ranks and iteration counts (Section 6.5) as transmitted second-hand and unverified. They have since been checked against [7] directly (its Sections 3.2.3–3.2.4): $K_i=10$, $K_r=20$, 100 and 60 multiplicative-update iterations and the STFT window are confirmed exact, and the check found two discrepancies that were corrected — the STFT hop had been an unverified 128 (the paper’s is 256), and the paper bandpass-denoises every snippet at 50–1800 Hz before the STFT, a stage the first reproduction lacked. Neither change moved Baseline 2’s full-set SDR by more than the fold-to-fold spread, so the method’s shortfall against Baseline 1 was not caused by either. The paper’s own training-dataset description remains ambiguous between ICBHI 2017+Fraivan and HLS-CMDS’s isolated recordings (Table 27); this project fits on HLS-CMDS’s isolated recordings under the leakage-safe pool and says so.
- **The oracle-mask ceiling is now measured, and it refutes the informal note that prompted it.** `oracle_mask_ceiling.py` runs three oracle masks over all 36 valid rows (Section 12); the ideal ratio mask reaches 11.5 dB heart SDR against 2.5 dB for the raw mixture, so there is no dataset-imposed ceiling near the no-separation floor. What the check does establish is a different ceiling: a magnitude mask on a 512-point STFT cannot exceed roughly 12 dB here, which is below Architecture 1’s knee region. Whether a finer grid or a time-domain method lifts that cap is untested.
- **No sensor-noise robustness claim.** As noted in Section 11.2, HLS-CMDS mixtures contain no independent noise source, so this pipeline’s SAR does not speak to robustness against real acquisition noise (relevant to hardware-deployed classifiers such as [8]’s).
- **Citation completeness.** Full bibliographic details (author, venue, year) for four of the six related-work papers were not available when this report was drafted ([7] has since been

resolved, Section 11); the remaining entries should be completed before any external submission.

- **Table 6 typography.** The related-work table is scaled to the page width via `resizebox` to fit six columns, which produces a few awkward line breaks inside narrow cells (e.g. “HLS-CMDS”, “SNR/SDR/correlation”). Cosmetic only, but worth widening manually (e.g. splitting into two tables, or rotating to landscape) before any polished submission.
- **No CI wiring.** The 31-test suite (Section 6.1.1) only runs when invoked manually (`make test`).
- **Small heart-sound classes threaten the planned classifier’s per-fold coverage.** Table 2 (Section 3.2.1) shows S4 ($n=2$), AV Block, and Tachycardia ($n=3$ each) are far smaller than the other heart classes. Under the leak-group-respecting 5-fold split, a 2-recording class cannot be represented in every fold’s training set; the classification stage (PROTOCOL.md §5.3) should decide up front whether to merge, exclude, or specially stratify these classes rather than discover the problem as an unexplained per-class accuracy anomaly later.

References

- [1] Y. Torabi, S. Shirani, and J. P. Reilly, “Descriptor: Heart and Lung Sounds Dataset Recorded from a Clinical Manikin using Digital Stethoscope (HLS-CMDS),” *IEEE Data Descriptions*, 2025. doi:10.1109/IEEEDATA.2025.3566012.
- [2] E. Vincent, R. Gribonval, and C. Févotte, “Performance measurement in blind audio source separation,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 14, no. 4, pp. 1462–1469, 2006.
- [3] C. Raffel et al., “mir_eval: A Transparent Implementation of Common MIR Metrics,” in *Proc. ISMIR*, 2014.
- [4] Y. Luo and N. Mesgarani, “Conv-TasNet: Surpassing Ideal Time–Frequency Magnitude Masking for Speech Separation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 27, no. 8, pp. 1256–1266, 2019.
- [5] Y. Torabi, “AI-Driven Cardiorespiratory Signal Processing: Separation, Clustering, and Anomaly Detection,” Ph.D. dissertation, McMaster University, 2025. (Cited via the Notion Reading List; not yet independently read from the primary source — see Section 13.)
- [6] A. Yaqub, M. S. Orakzai, M. F. Qureshi, Z. Mushtaq, I. Siddique, and T. Radwan, “Spectrotemporal Deep Learning for Heart Sound Classification under Clinical Noise Conditions,” *Computer Modeling in Engineering & Sciences*, vol. 145, no. 2, pp. 2503–2529, 2025. doi:10.32604/cmes.2025.071571. (Independently read from the primary source — see Section 11.)
- [7] Y. Han, Z. Quan, K. Matuszewski, and B. Corbett, “Respiratory Disease Classification Using NMF-Enhanced Log-Mel Spectrograms and Convolutional Recurrent Neural Networks,” *Sensors*, vol. 26, no. 13, art. 4268, 2026. doi:10.3390/s26134268. (Independently read from the primary source.)
- [8] A. Puneet, P. Shankar, S. R. R. Koluguri, and A. Srivastava, “Edge-Enabled Portable Classifier for Lung Sounds Using Convolutional Neural Networks,” in *Proc.*

2025 IEEE Biomedical Circuits and Systems Conference (BioCAS), pp. 21–25, 2025. doi:10.1109/BioCAS67066.2025.00016. (Independently read from the primary source.)

- [9] B. Han and W. Quan, “Cardiorespiratory Sound Separation Using Singular Spectrum Analysis,” in *Proc. 2025 17th Int’l Conf. on Signal Processing Systems (ICSPS)*, 2025. doi:10.1109/ICSPS66615.2025.11347745. (Independently read from the primary source.)